MZ IPTV Player — Documentation

Oct 26, 2026

MZ IPTV Player is a live-TV player for Android TV, Android phones and tablets, and web browsers. One web page provides the whole interface; on Android, a native ExoPlayer shell plays the video underneath it.

0. About this document

Field	Value
Product	MZ IPTV Player
Product version	0.1.0
Package	`com.mazem3dia.iptv`
Repository	MazeM3dia/MZ-IPTV-Player
Licence	MIT, © 2026 Maze Media
Document status	Draft for review

0.1 Audience

Reader	Start at
End users watching TV	Part 1, User Manual
New contributors learning the code	Parts 2 and 3
Maintainers changing playback, input or the native shell	Parts 4 and 5
Anyone looking up a name, key or constant	Part 6, Reference

0.2 Organisation

The parts follow the Diátaxis split of documentation by purpose:

- **Manual** (Part 1): task-oriented use of the app.
- **Explanation** (Parts 2–4): how and why the code works, from basics to advanced techniques.

- **How-to** (Part 5): building, testing, releasing and extending.
- **Reference** (Part 6): tables of APIs, keys, data shapes and terms.

0.3 Conventions

- `code` marks identifiers, file names, keys and commands.
- `file:line` references point into the repository at version 0.1.0; line numbers drift as code changes, so search for the identifier if a line has moved.
- **Page** means `web/index.html`. **App** means the Android build. **Browser** means the page opened directly in a web browser.
- **Native** means Kotlin code in `MainActivity.kt`; `NATIVE` in capitals is the JavaScript flag of that name.
- Key names in the manual use the remote label first, then the keyboard key, e.g. OK / Enter.

Part 1. User Manual

The app opens with channels already loaded: the free iptv-org playlist downloads by itself, and your last channel resumes. You only browse and press OK.

1.1 About the app

MZ IPTV Player plays live TV channels listed in an M3U playlist. It works like a TV set: channels have numbers, the last one you watched starts when you turn it on, and the remote does everything.

What it does

- Loads the free iptv-org playlist (about 11,000 public channels from around the world) automatically, or any M3U playlist you give it.
- Plays HLS (`.m3u8`), DASH (.mpd) and plain HTTP streams (TS, MP4); DASH needs the Android app.
- Shows each channel's country, resolution, genre and availability hints.
- Remembers favourites, recently watched channels, filters and the last channel.

What it does not do

- It does not host, store or supply any streams. Every channel is a link from the playlist you load.
- It does not record, rewind or show a programme guide (EPG). Live channels play at the live moment only.

- It does not get around geo-blocking or logins. Channels that refuse your connection show an error card with the reason.
- It does not play copy-protected (DRM) channels, which many paid DASH services use.

> **Content notice.** The app shows this on its welcome card: *"This player only plays links you provide — it doesn't host, store, or supply any streams. You're responsible for what you load."*

1.2 System requirements

Platform	Minimum	Input	Notes
Android TV / Google TV	Android 9; Android System WebView 62+ (Android 9 ships 66)	TV remote, game controller	Recommended way to watch
Android phone or tablet	Android 9; Android System WebView 62+ (Android 9 ships 66)	Touch	Same APK as TV
Web browser	A current browser; tested in Chrome, Edge and Firefox, and in WebKit (Safari's engine) for layout and controls	Keyboard, mouse, touch	Some channels play only in the app (see 1.12)
Network	A stable broadband connection	—	Far-away servers buffer more

The page has fallbacks for older WebViews, so the WebView an Android 9 box shipped with is enough. Only below version 62 does the app open a dialog, once per WebView version; its *Update* button appears only when the Play Store can update that WebView. You can still choose *Continue*, but the screen may not look right.

1.3 Installing, updating and uninstalling

The app is not on Google Play yet. You install it from an APK file, which Android calls *sideloading*.

Android phone or tablet

1. On the phone, open the repository's **Releases** page and download `MZ-IPTV-Player-<version>-release.apk`.
2. Open the downloaded file from the notification or the *Files* app.

3. If Android asks, allow **Install unknown apps** for the app you opened it from (Chrome, Files and so on), then go back.
4. Tap **Install**. If Google Play Protect warns that the app is unknown, tap **More details → Install anyway**.
5. Open **MZ IPTV Player** from the app drawer.

Android TV or Google TV

Pick one of three ways to get the APK onto the TV:

Method	Steps
USB stick	Copy the APK to a USB stick, plug it into the TV, open it with a file manager app, allow unknown apps for that file manager, install
Downloader app	Install *Downloader* from the TV's app store, enter the APK's download address, install when the download ends
`adb` from a computer	Turn on *Developer options → USB / network debugging* on the TV, then run `adb install MZ-IPTV-Player-<version>-release.apk`

The app then appears in the TV's **Apps** row with its banner.

Web browser

1. Download or clone the repository.
2. Open `web/index.html` in the browser, from disk or from any web server.
3. Keep the `web/vendor/` and `web/icons/` folders next to `index.html`; the page needs them.

Updating

Install the newer APK over the old one. Favourites, history, filters and your playlist stay. If Android says the package conflicts with an existing one, the new APK was signed with a different key: you must uninstall the old app first, which deletes your saved data.

Uninstalling

- **Phone or tablet:** long-press the app icon → **Uninstall**, or *Settings → Apps → MZ IPTV Player → Uninstall*.
- **TV:** *Settings → Apps → MZ IPTV Player → Uninstall*.

- **Browser:** delete the folder. To clear saved favourites and history, clear the site data for that page in the browser settings.

1.4 First start

You do not need to type anything on first start. The app loads the default playlist and opens a welcome card that lists the controls for your device.

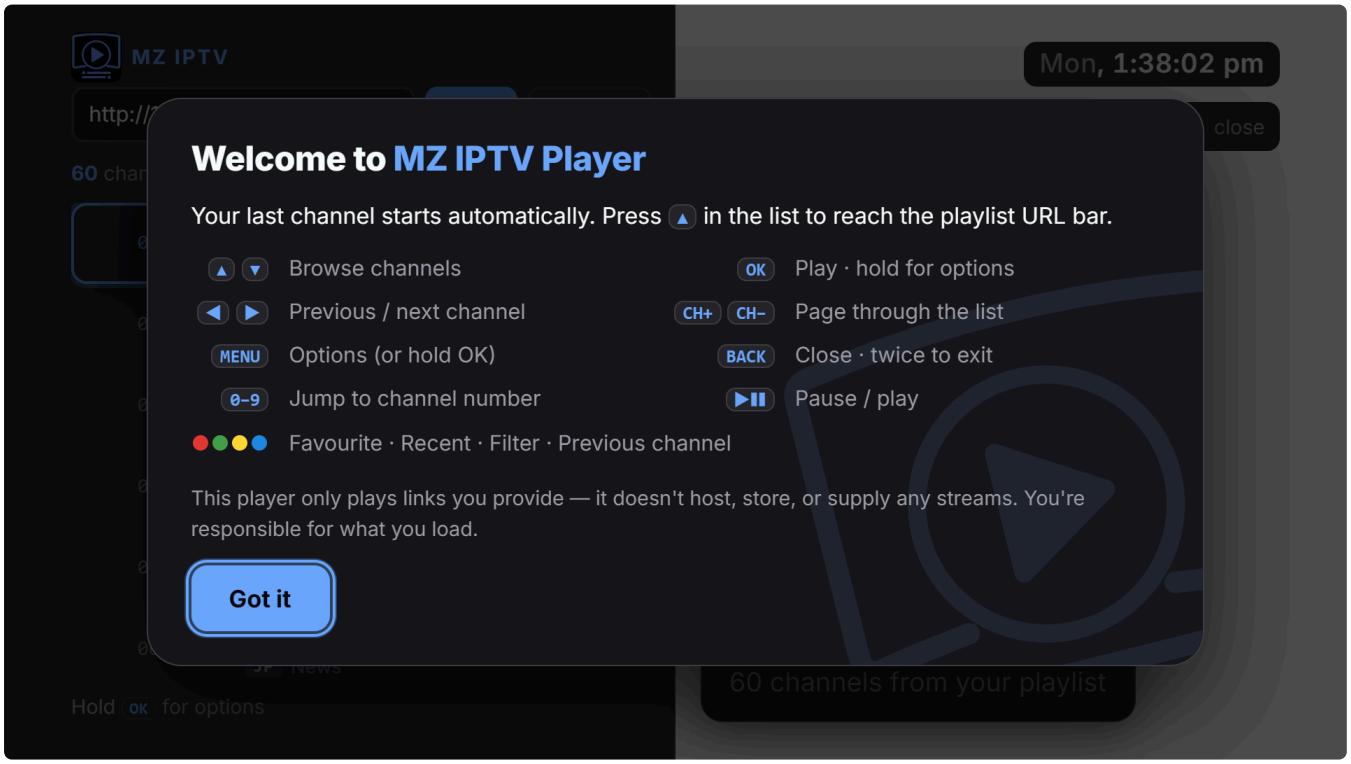


Figure 1.1 — The welcome card on a TV. In a browser it lists keyboard keys; on a touch screen it lists gestures.

1. The app downloads the default playlist, `https://iptv-org.github.io/iptv/index.m3u`. This takes a few seconds; the list shows *Loading playlist…* meanwhile.
2. A banner reports how many channels loaded, e.g. *11,302 channels from the default playlist*.
3. The welcome card appears on top. Read it, then press **OK**, **Enter**, **Back** or tap **Got it** to close it.
4. The channel list opens. Pick a channel and press **OK** (or tap it) to start watching.
5. From now on, every start resumes the last channel you watched and keeps the list closed, like turning on a TV.

The welcome card appears only once. To see it again, open the options menu (hold **OK**) and choose **Remote guide** (TV) or **Controls guide** (browser and touch).

1.5 A tour of the screen

The app has two layouts. **Landscape** (TV, computer, phone held sideways) shows full-screen video with a channel list that slides in from the left. **Portrait** (phone held upright) shows the video on top and the list below.

The channel list (landscape)

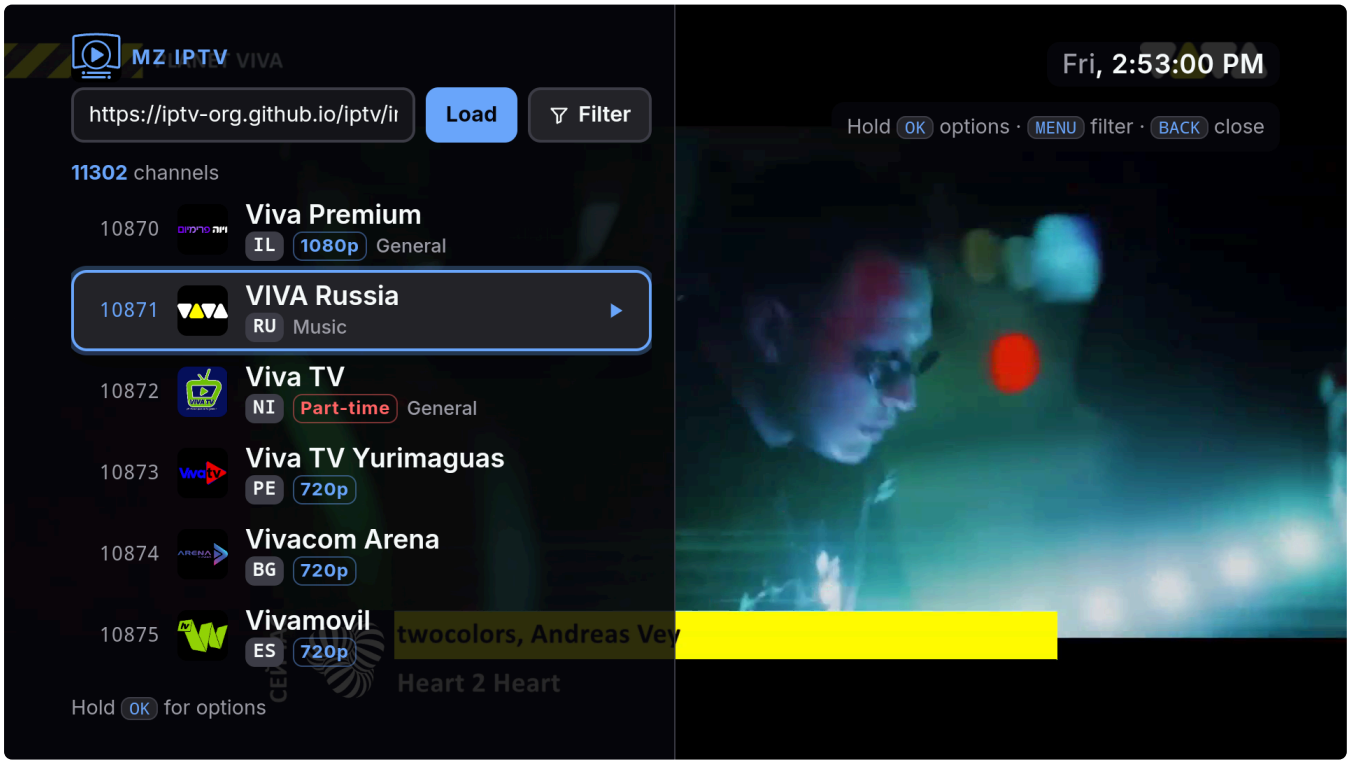


Figure 1.2 — The channel list on Android TV, with the playing channel focused.

Part	What it shows
Logo, top left	App name
URL field, **Load**, **Filter**	The playlist address, a button to load it, and the filter panel. Browsers also show **File**
Count line	*11302 channels*, or *10 matches · United States* when a filter is on
Channel row	Number, logo, name, then badges: country, resolution, *Part-time* or *Geo*, genre
Blue outline	The focused row, where **OK** acts
▶ at the row's right	The channel that is playing
Footer	*Hold OK for options*

Part	What it shows
Clock and hint, top right	Day and time, and the main shortcuts. Both fade after 5 seconds without input

The list closes by itself after 8 seconds without input, so the picture is not covered. Press ▲, ▼ or **OK** to bring it back.

> The hint chip in the real-device screenshots says *MENU filter*; current builds show *Hold OK options · BACK close*, and **MENU** opens the options menu.

Now playing

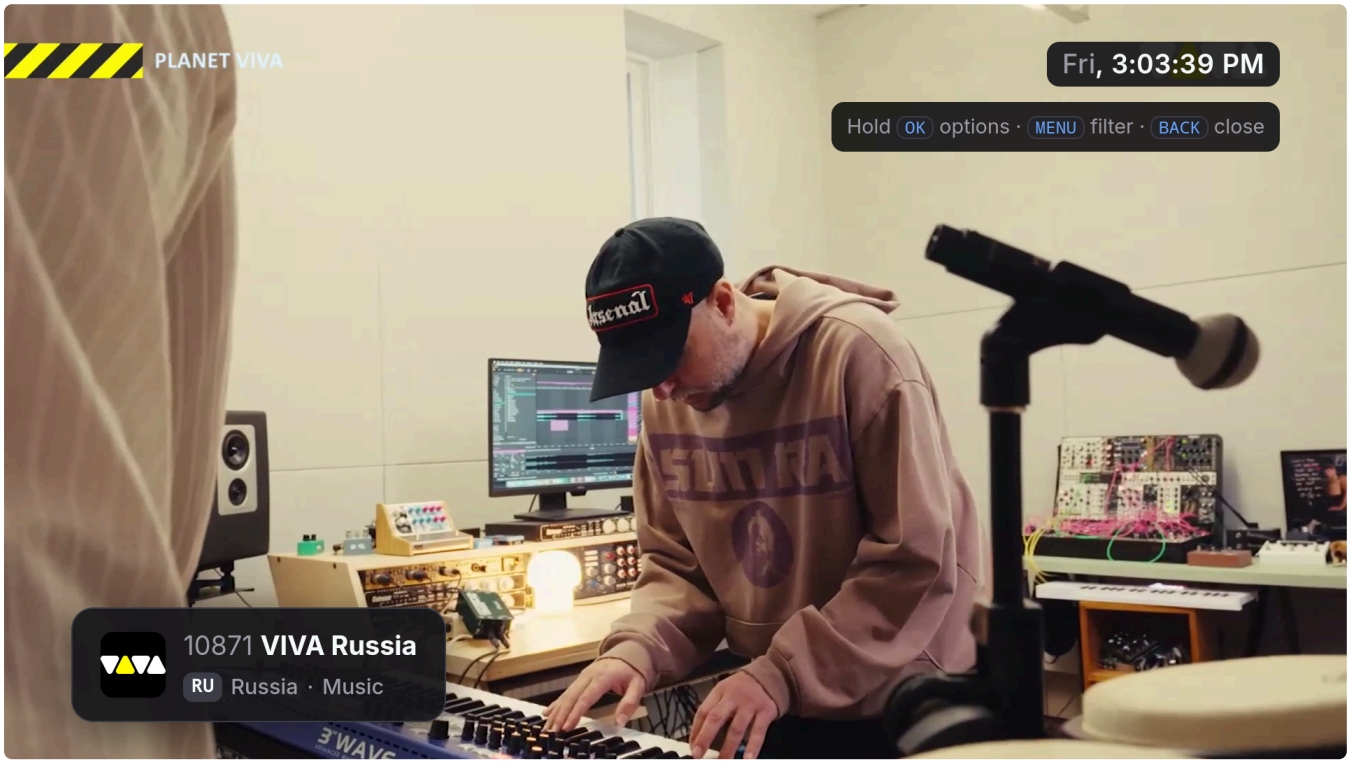


Figure 1.3 — After tuning, a banner shows the number, name, logo, country and genre for 4 seconds.

Press **Info** or **I** at any time to show the banner again.

Phone, portrait

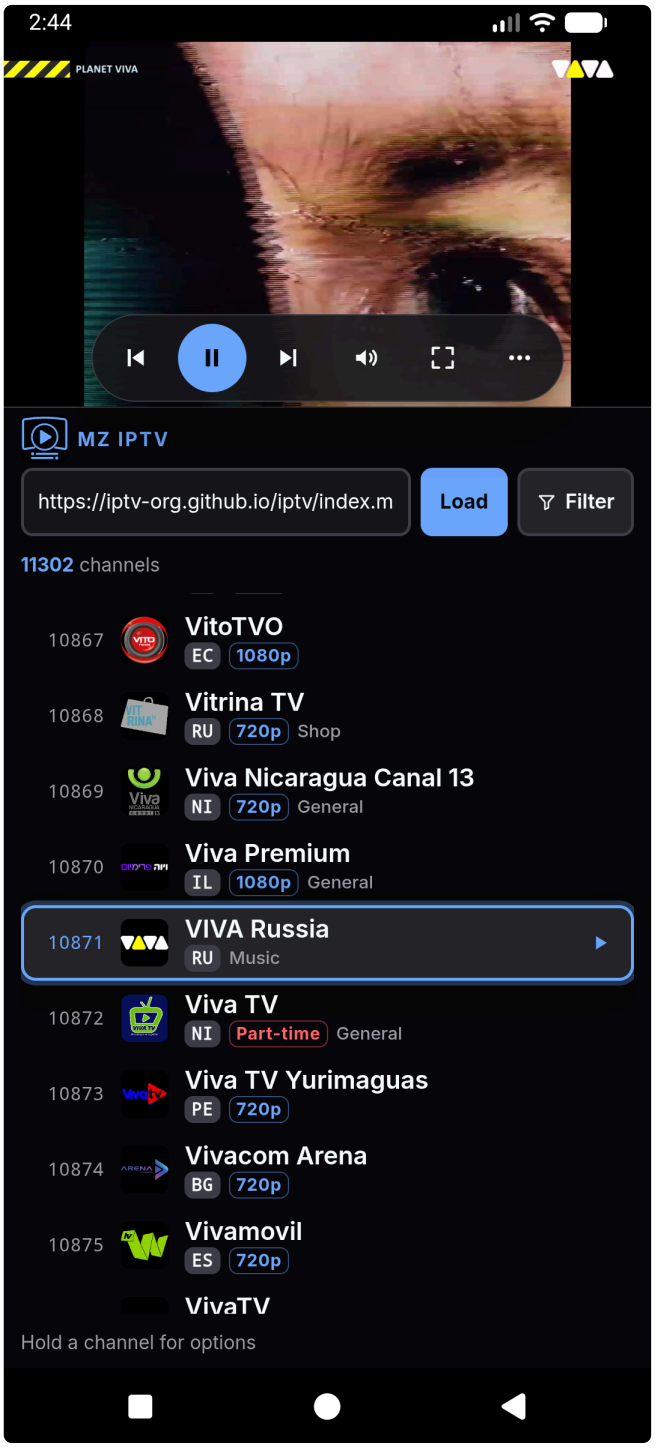


Figure 1.4 — Portrait on a phone. The status and navigation bars stay visible; the list never hides.

Browser

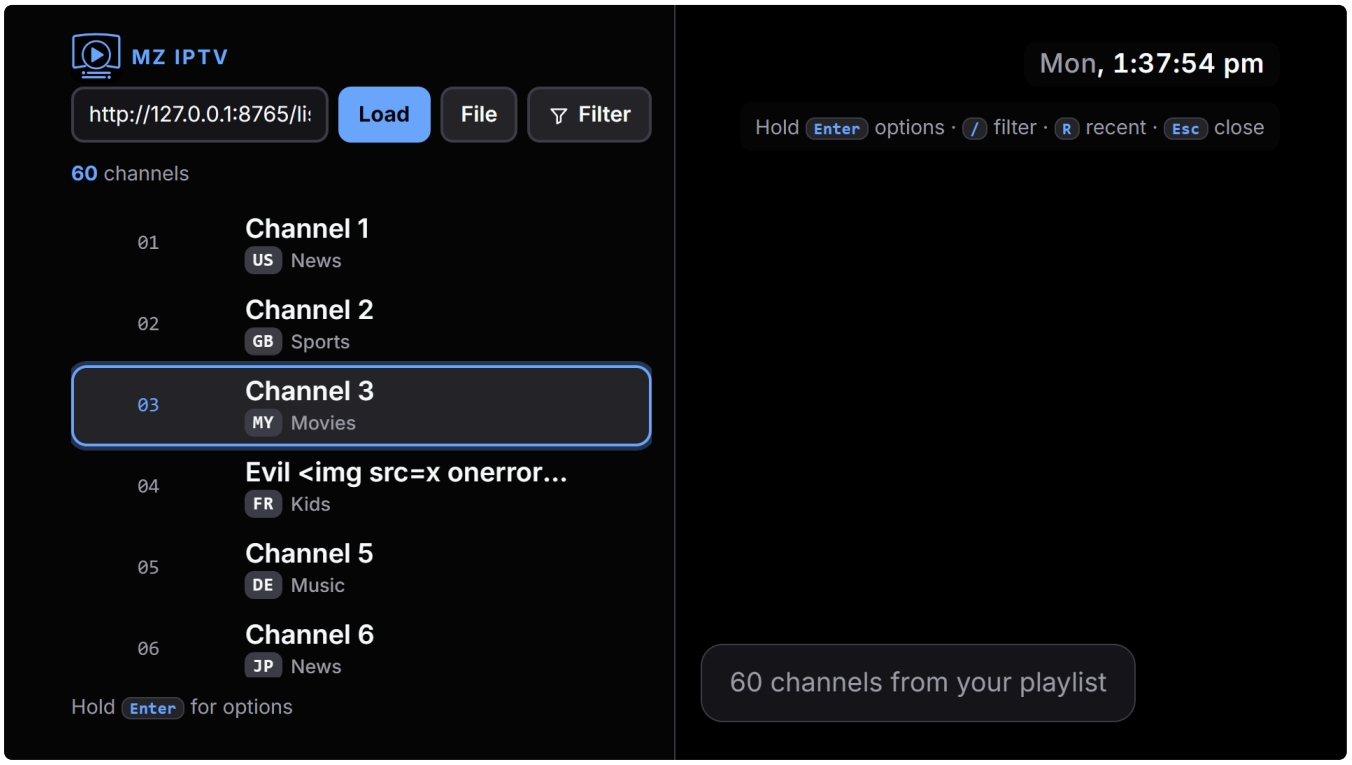


*Figure 1.5 — The browser version (test playlist). Hints show keyboard keys, and a **File** button opens a local playlist.*

1.6 Controls

Every feature works from every input. The TV remote is the main design target; keyboard keys mirror it, and touch gets buttons and gestures.

TV remote

Button	In the channel list	With the list closed
▲▼	Move focus; hold to speed up; ▲ on the first row reaches the top bar	Open the list
OK	Play the focused channel	Open the list (retry, if the channel failed)
Hold OK	Options menu for the focused channel	Options menu for the playing channel
◀▶	Previous / next channel	Previous / next channel
CH+ CH−	Page down / up through the list	Next / previous channel
0–9	Type a channel number	Type a channel number

Button	In the channel list	With the list closed
BACK	Close the list	Press twice within 2.5 s to exit
MENU	Options menu	Options menu
▶❚❚, Play, Pause, Stop	Pause / resume	Pause / resume
Info	Show the now-playing banner	Same
Guide	Open / close the list	Same
Search	Filter panel with the search field focused	Same
Red · Green · Yellow · Blue	Favourite · Recently watched · Filter · Previous channel	Same
Bookmark	Favourite	Same
Rewind / Fast-forward	Previous / next channel	Same

Game controllers work too: **A** = OK (hold for options), **B** = Back, **Start** = pause, **L1 / R1** = previous / next channel.

Inside panels and menus

Button	Filter panel	Country / Category picker	Options menu, Recently watched
▲▼	Move between rows	Move; hold to speed up	Move
◀▶	Change Country or Category without opening the picker	◀ closes the picker	—
CH+ CH−	—	Jump 8 rows	—
OK	Toggle, open a picker, or run the row	Choose	Run the item / play the channel
BACK	Close the panel	Back to the filter panel	Close

Keyboard and mouse (browser)

Key	Action
↑ ↓	Browse channels
Enter	Play · hold for options
← →	Previous / next channel
PgUp PgDn	Page through the list (change channel when it is closed)
/	Filter and search
Esc	Close
0–9	Type a channel number
Backspace	Delete a typed digit, otherwise previous channel
Space	Pause / play
F	Favourite
R	Recently watched
I	Show the now-playing banner
M	Mute / unmute

With a mouse: click a channel to play it; right-click a channel for its options; scroll the list with the wheel; drag a `.m3u` file onto the page to load it. Shortcuts with Ctrl, Alt or Cmd are left to the browser.

Touch (phone, tablet, touch-screen browser)

Gesture	Action
Tap a channel	Play it
Hold a channel	Options for that channel
Swipe ← → on the video	Next / previous channel
Tap the video	Show or hide the control bar; in landscape, also closes the list
Tap outside a menu	Close it

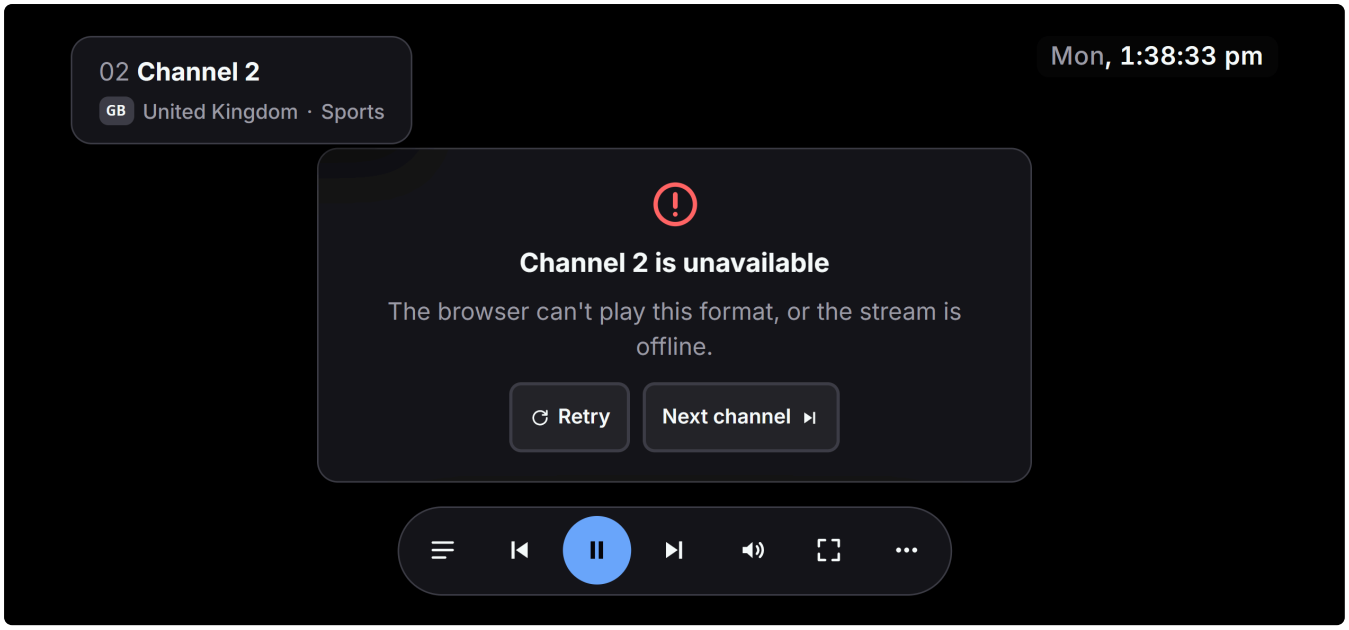


Figure 1.6 — The touch control bar in landscape, here under an error card with Retry and Next channel buttons.

Control bar button (left to right)	Action
List	Open the channel list (landscape only)
Previous	Previous channel
Play / Pause	Pause or resume
Next	Next channel
Speaker	Mute / unmute
Expand	Fullscreen; in the Android app, turns the picture to landscape
…	Options menu

1.7 Watching TV

Browse and play

1. Press ▲ or ▼ to open the list and move the blue outline. Hold the button to scroll faster: speed rises from 1 row to 3 rows, then 10 rows per step.
2. Press **CH+** / **CH−** (PgDn / PgUp) to jump a page at a time.
3. Press **OK** on a channel. The list closes and the screen shows *Tuning ‹name›…*, then *Buffering…* while the stream starts.
4. When the picture appears, the now-playing banner shows for 4 seconds.

Pressing **OK** on the channel that is already playing just closes the list. On a channel that failed, it tries again.

Change channel

- ◀ / ▶ (or swipe on the video, or Previous / Next on the control bar) moves to the previous / next channel in the list.
- The order wraps around: after the last channel comes the first.
- Channel changes stay inside the current filter. With *Country: Japan* on, ▶ moves only between Japanese channels.

Jump to a channel number

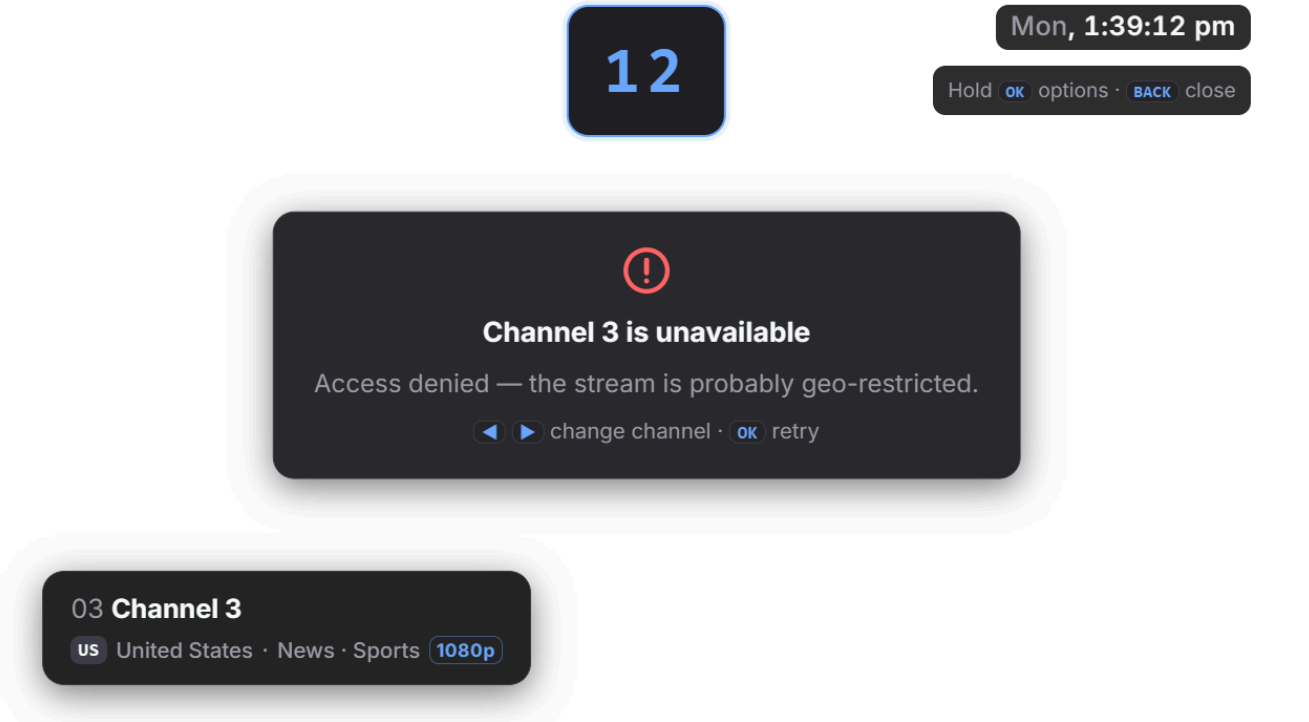


Figure 1.7 — Typing 1, 2 shows the number at the top. The app tunes 1.3 seconds after the last digit.

1. Type the digits with the remote or keyboard, up to 6 digits.
2. Wait 1.3 seconds, or press **OK** to tune at once.
3. **Backspace** removes the last digit; **Back** cancels.

Numbers are positions in the list *as currently shown*. With a filter on, channel 1 is the first channel that matches. A number past the end shows *No channel ‹number›*.

Previous channel

Press **Blue**, **Last channel**, **Backspace**, or choose *Options* → *Previous channel* to jump back to the channel you watched before this one. Press it again to flip back. It must be in the current filter; otherwise the app says *Previous channel isn't in the current filter*.

Pause and resume

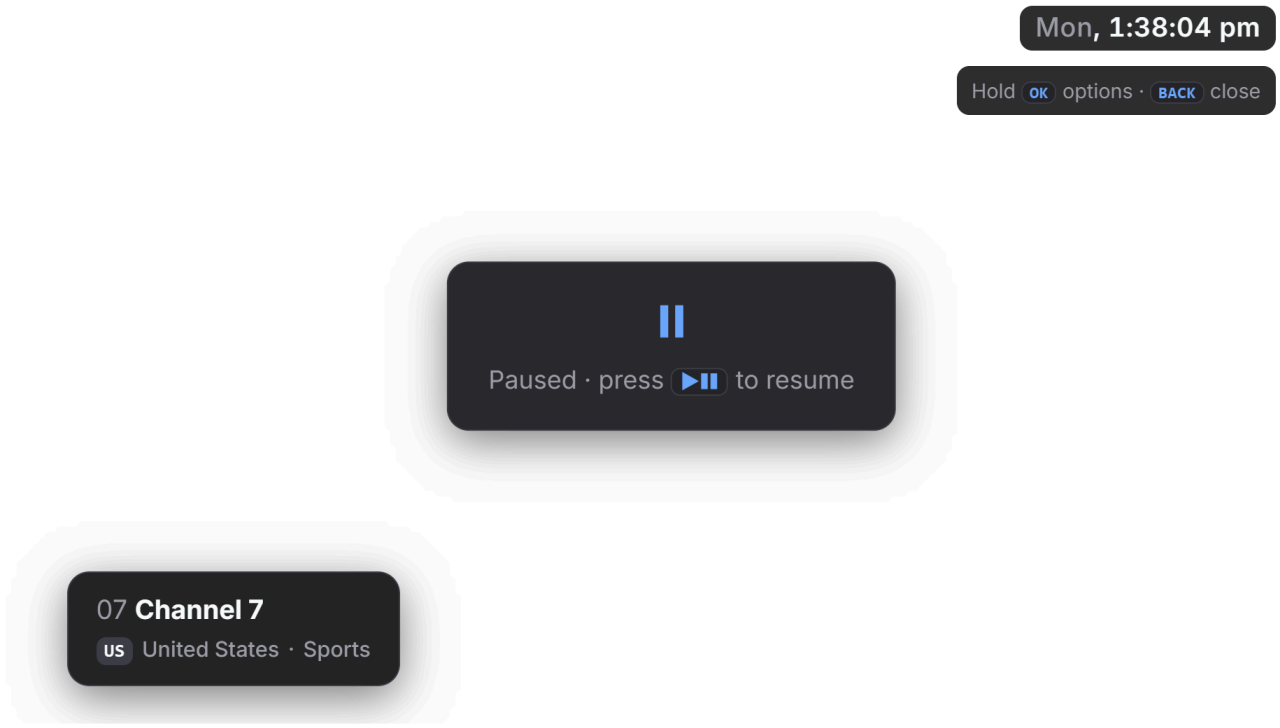


Figure 1.8 — The paused card.

Press ▶❙❙ (Space, or the Play/Pause button) to pause. Because this is live TV, resuming jumps to the *live* moment, not to where you paused.

Sound

Press **M**, the speaker button, or *Options* → *Mute* to mute or unmute. A banner confirms *Muted* or *Sound on*. TV volume itself stays on the TV remote's volume buttons.

Channel information

Press **Info** (or **I**) to show the now-playing banner again. If nothing is playing it says *Nothing playing yet*.

1.8 Finding channels: filter and search

The filter panel narrows 11,000 channels down to the ones you want. Filters combine: *Japan* + *News* + *Favourites only* shows only your favourite Japanese news channels.

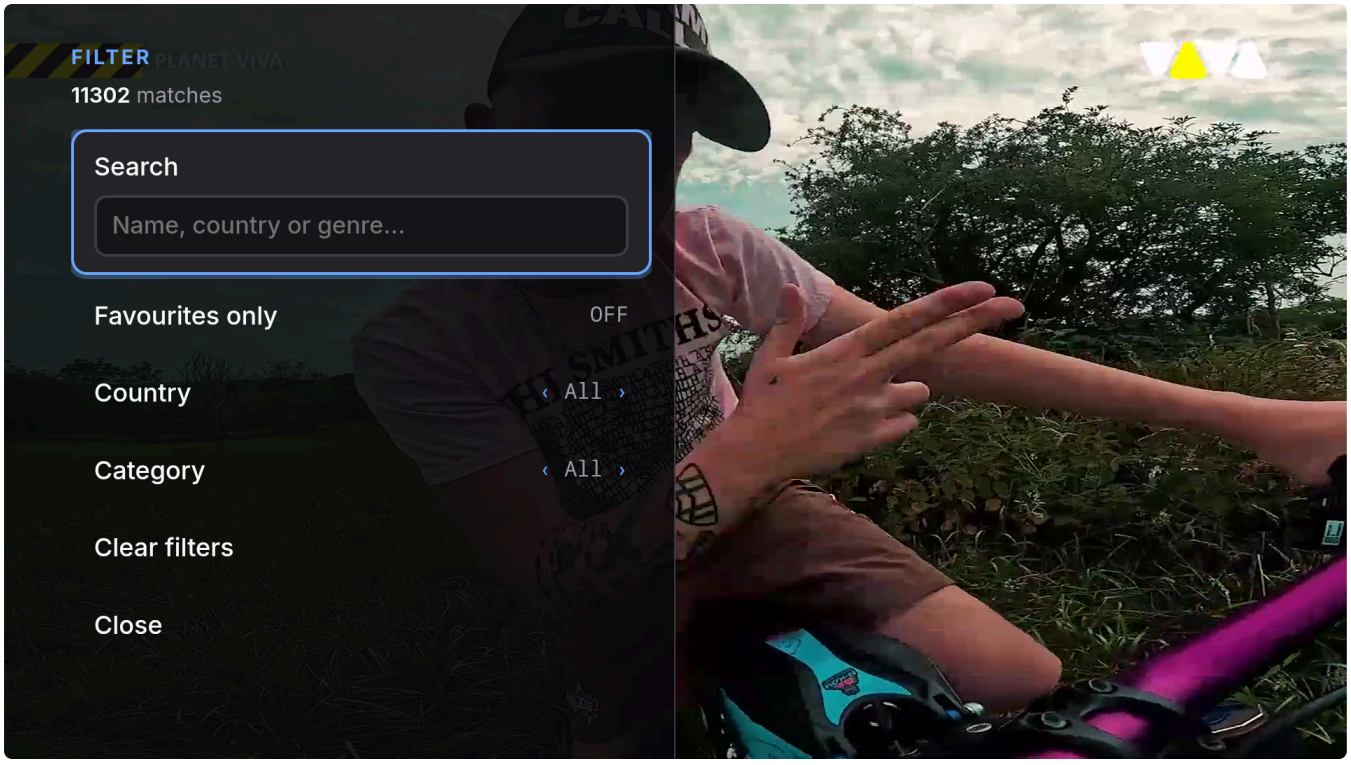


Figure 1.9 — The filter panel. The line under the title counts the matching channels.

Open the panel

Device	How
TV remote	**Yellow**, or hold **OK** → *Filter & search*; **Search** opens it with the search field ready
Keyboard	**/**
Mouse or touch	The **Filter** button in the top bar, or … → *Filter & search*

In landscape the channel list hides while the panel is open, so the panel has the full height.

Rows of the panel

Row	What it does	How to change it
Search	Matches channel name, country name or genre as you type	**OK** to start typing (on TV, the on-screen keyboard opens); ▼, **Enter** or **Back** to leave the field
Favourites only	Shows only starred channels	**OK** switches ON / OFF

Row	What it does	How to change it
Country	Limits to one country	◀▶ steps through countries; **OK** opens the picker
Category	Limits to one genre (News, Kids, Music …)	◀▶ or **OK**, as for Country
Clear filters	Resets all four filters at once	**OK**
Close	Closes the panel; filters stay on	**OK**, or press **Back** anywhere in the panel

The Country and Category pickers

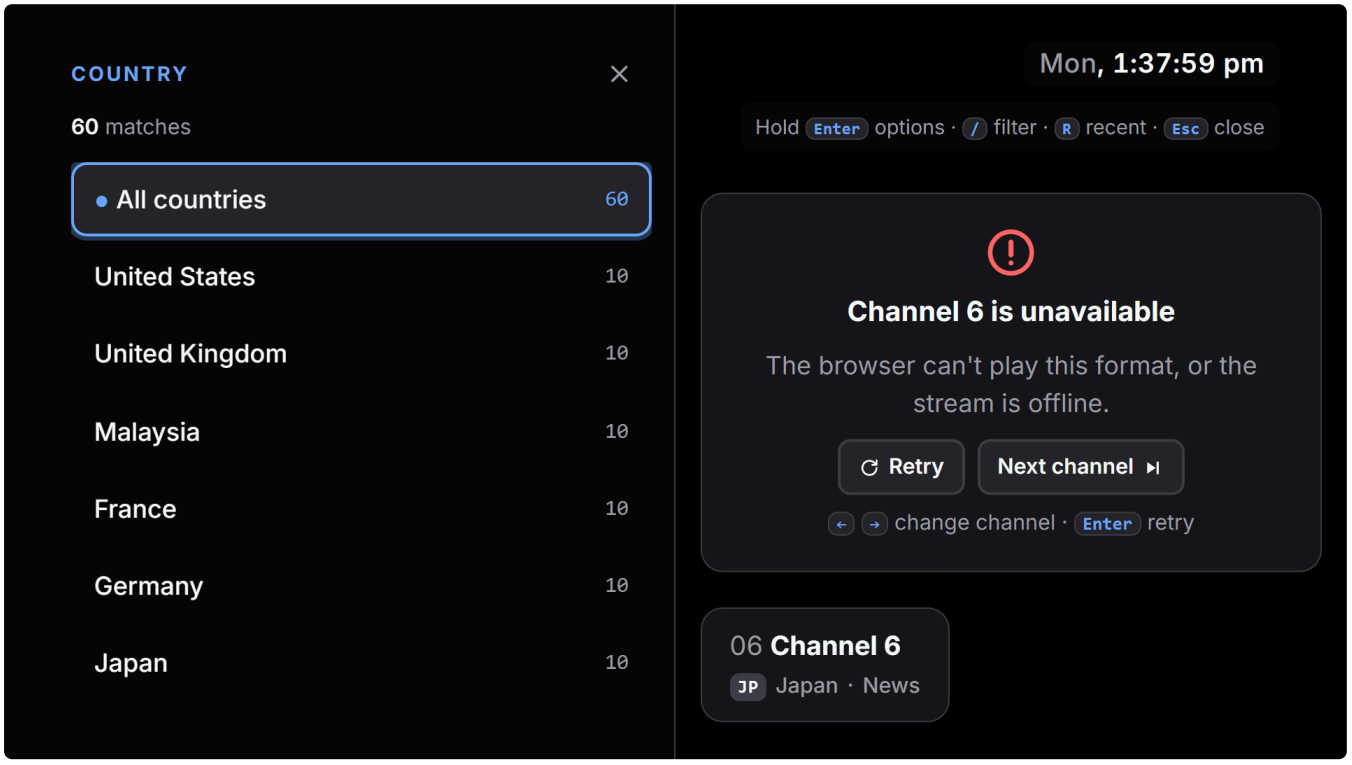


Figure 1.10 — The country picker (test playlist). Each row shows how many channels it has; the dot marks the current choice.

- Countries and categories are sorted by channel count, largest first, so the busiest choices are at the top.
- Hold ▼ to scroll faster, or use **CH+ / CH−** to jump 8 rows.
- **OK** chooses; **Back** or ◀ returns to the panel without changing anything.

Tips

- The count line above the list shows the active filters, e.g. *10 matches · “news” · United States*.
- Filters are saved and still apply after a restart.

- If nothing matches, the list says *No channels match these filters*; press **OK** (or tap **Clear filters**) to reset.
- The playing channel keeps its place: after a filter change, focus stays on it when it still matches.

1.9 Favourites and recently watched

Favourites

Star the channels you watch often, then turn on *Favourites only* in the filter panel to see just those.

To	TV remote	Keyboard	Touch / mouse
Star or unstar	**Red** or **Bookmark**	**F**	Hold (or right-click) the channel → *Add to favourites*
Star via menu	Hold **OK** → *Add to favourites* / *Remove from favourites*	Hold **Enter**	… → *Add to favourites*
Show only favourites	Filter panel → *Favourites only: ON*	**/** → same	**Filter** → same

- With the list open, the star applies to the **focused** channel; with the list closed, to the **playing** channel.
- A banner confirms *Favourited ‹name›* or *Removed ‹name›*, and starred rows show a star before the name.
- Favourites survive playlist refreshes and app updates, because they are tied to the channel's ID rather than its stream address.

Recently watched

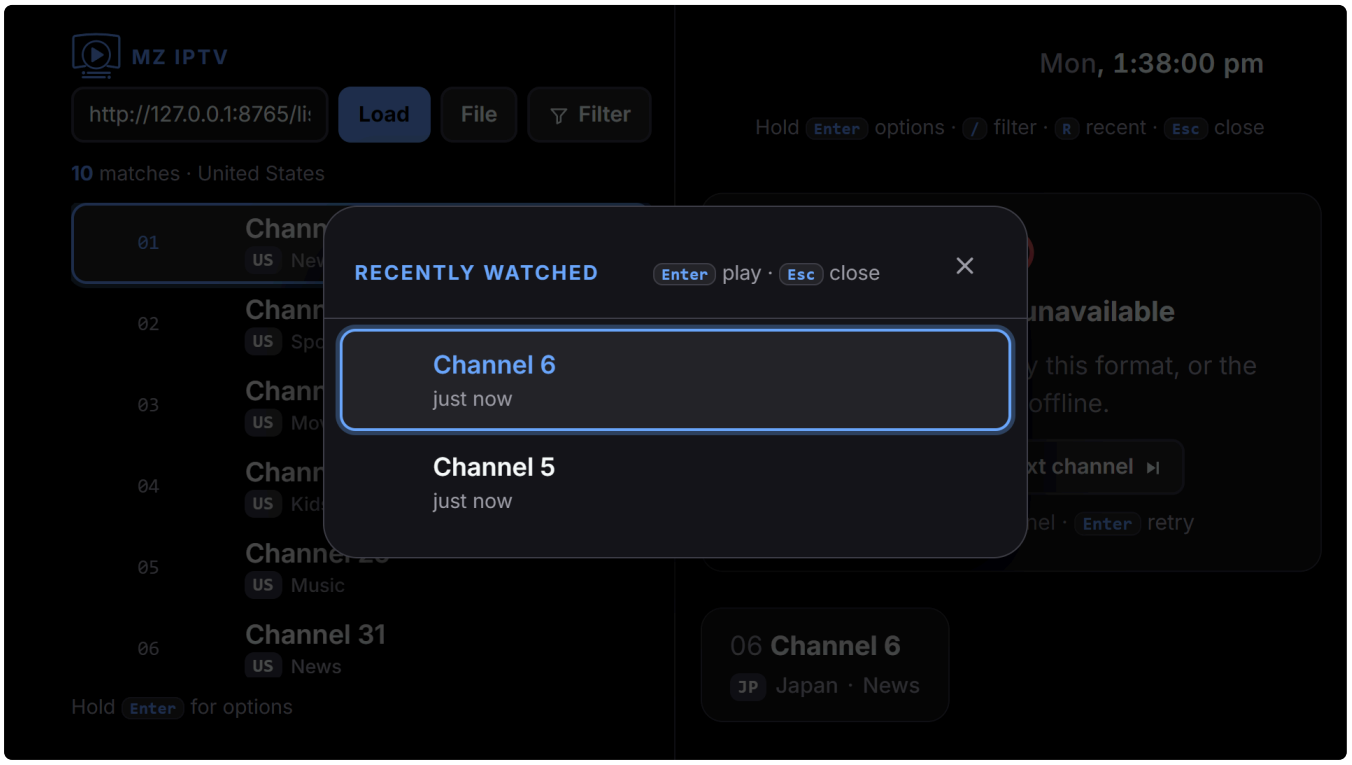


Figure 1.11 — Recently watched (test playlist), newest first, with how long ago each was watched.

1. Press **Green** (TV), **R** (keyboard), or choose *Options → Recently watched*.
2. Move with ▲▼ and press **OK** to play a channel, or tap it.
3. Press **Back**, **Green** or **R** again to close.

The list keeps your last 15 channels. Times read *just now*, *5 mins ago*, *2 hrs ago* or *3 days ago*. If a recent channel is hidden by the current filter, the app clears the filter and plays it. If it is no longer in the playlist, the app says *That channel isn't in the current playlist*.

1.10 The options menu

The options menu holds every action that has no button of its own. Its title is the channel it acts on.

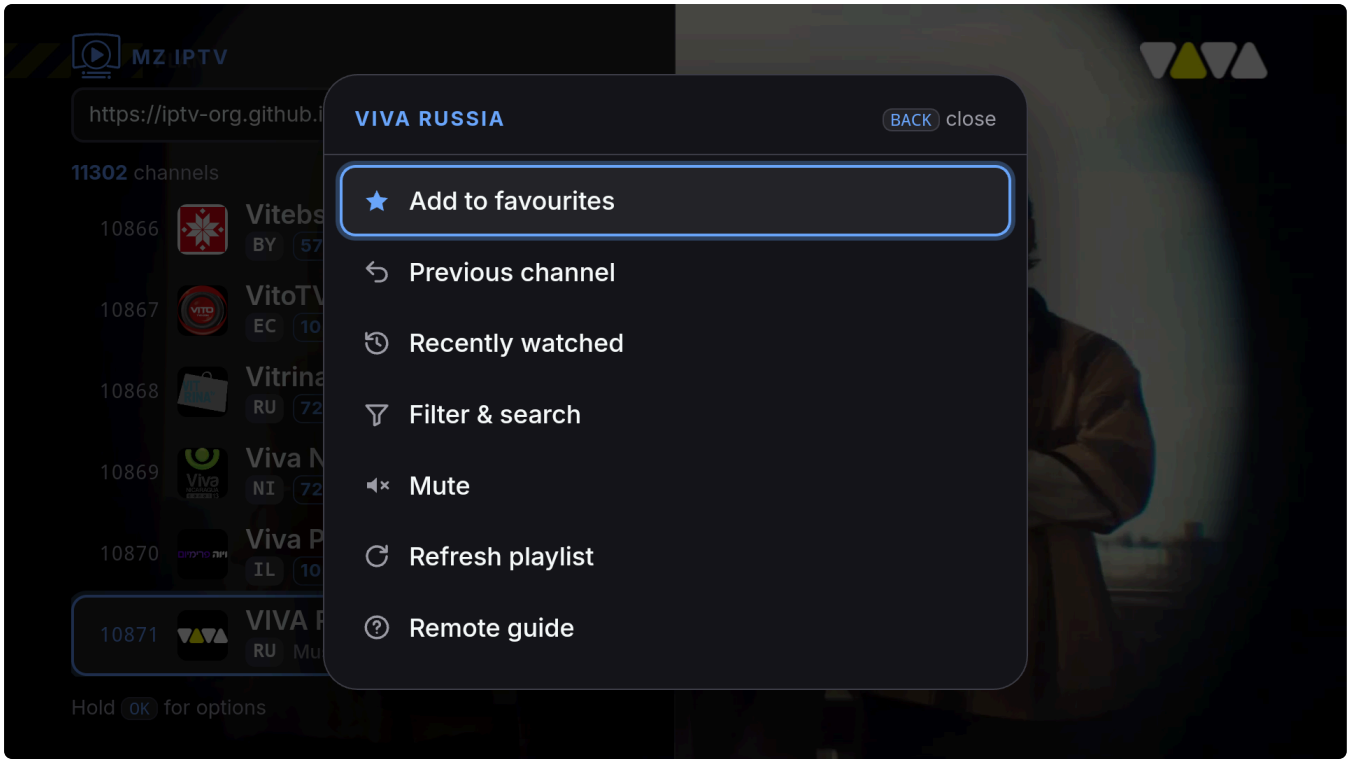


Figure 1.12 — The options menu for a channel.

Open it: hold **OK** (TV) or **Enter** (keyboard) for about half a second, press **MENU**, right-click or hold a channel, or tap **…** on the control bar. Close it with **Back**, **MENU** again, or a tap outside.

Item	Shown when	What it does
Add to favourites / Remove from favourites	A channel is focused or playing	Stars or unstars that channel
Retry this channel	The playing channel failed	Tunes it again
Previous channel	Always	Jumps to the channel you watched before
Recently watched	Always	Opens the last 15 channels
Filter & search	Always	Opens the filter panel
Mute / Unmute	Always	Turns the sound off or on
Refresh playlist	Always	Downloads the playlist again now, without stopping playback
Load default playlist	You are using your own playlist	Switches back to the iptv-org playlist

Item	Shown when	What it does
Remote guide / Controls guide	Always	Shows the welcome card with the controls again

Which channel is "this channel": with the list open, the focused one; with the list closed, the one playing.

1.11 Playlists

A playlist is a text file (M3U or M3U8) that lists channels and their stream addresses. The app starts with the iptv-org playlist and can load any other.

The default playlist

`https://iptv-org.github.io/iptv/index.m3u` is maintained by the iptv-org community and rebuilt daily. It lists publicly available channels only. Many of them are regional, part-time or geo-blocked, which is why some will not play where you are (see 1.12).

Load your own playlist from a URL

1. Open the channel list and press ▲ on the first channel to reach the top bar (or click / tap the URL field).
2. With the URL field focused, press **OK**. The on-screen keyboard opens on TVs and phones.
3. Type or paste the playlist address, starting with `http://` or `https://`.
4. Press **Enter**, or move right to **Load** and press **OK**.

The app remembers your playlist and loads it at every start. If it cannot be loaded at start-up, the app shows the default playlist instead with the message *Couldn't load your playlist (…) — showing the default one*, and tries yours again next time.

Load a playlist file (browser only)

- Click **File** in the top bar and pick a `.m3u` or `.m3u8` file, or drag the file onto the page.
- Files under about 1.5 MB are remembered and reopen by themselves. Larger files must be opened again after a restart; the default playlist loads meanwhile.

Go back to the default

Leave the URL field empty and press **Load**, or choose *Options → Load default playlist*.

Keeping the playlist fresh

- The playlist downloads again at every start, so you always begin with the current list.

- While the app stays open, it re-downloads in the background when the copy is older than 6 hours. It checks every 15 minutes and when you return to the app.
- A background refresh never interrupts playback, and waits while you are browsing the list or a menu.
- A banner reports the result when something changed, e.g. *Playlist updated · 11,310 channels (+8)*.
- *Options → Refresh playlist* forces a refresh at any time.

Channel badges

The app reads extra information from the playlist and shows it as badges in each row and in the now-playing banner.

Badge	Meaning	Where it comes from
`JP`, `US`, `GB` …	Broadcast country. Hover (mouse) to see the full name	The end of the channel's `tvg-id`, e.g. `NHKWorld.jp`
`720p`, `1080p`, `4K`, `HD`	Picture resolution	`(1080p)` in the channel name, or an `@HD` feed
Geo	The broadcaster limits the stream to some countries	`[Geo-blocked]` in the name
Part-time	The channel is not on air around the clock	`[Not 24/7]` in the name
News, Kids, Music …	Genres	The playlist's `group-title`

Making your own playlist

Any text editor can make a playlist. Each channel is two lines, an `#EXTINF` line and the stream address:

```
#EXTM3U
#EXTINF:-1 tvg-id="MyChannel.us" tvg-logo="https://example.com/logo.png"
group-title="News",My Channel (1080p)
https://example.com/live/index.m3u8
```

Optional extras the app understands: `http-user-agent="…"` and `http-referrer="…"` attributes, or `#EXTVLCOPT:http-user-agent=…` lines, for streams that need them (Android app only).

1.12 Messages and status cards

The app talks to you in two ways: a **status card** in the middle of the screen while a channel is starting, paused or failed, and a short **banner** near a corner that disappears after 4 to 7 seconds.

When a channel does not play

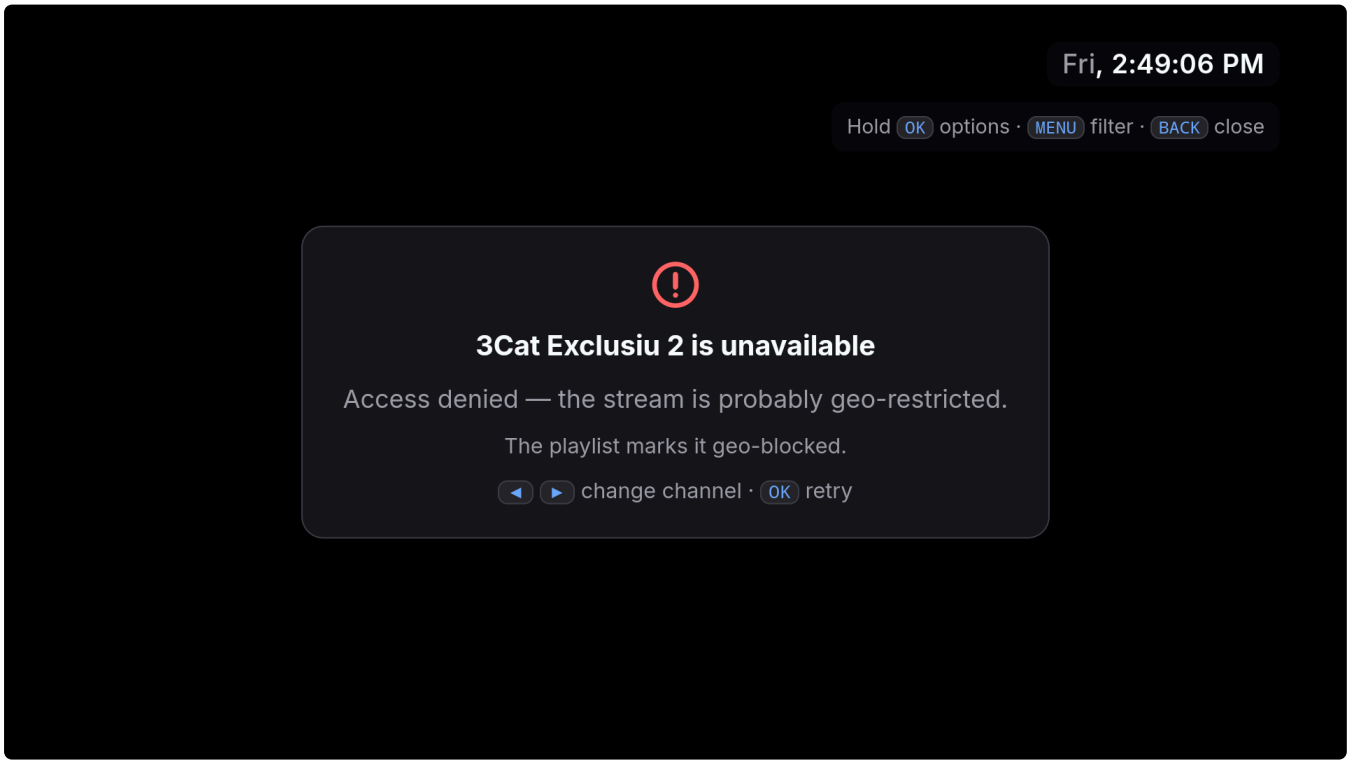


Figure 1.13 — An error card. The second line is the reason; the third line repeats what the playlist says about the channel.

Many free channels are geo-restricted, offline or part-time, so error cards are normal. The app tells you why and never tries to get around a restriction. The card stays until something plays.

- Press ◀ / ▶ to move on, or **OK** to retry. Touch screens show **Retry** and **Next channel** buttons.
- *Options → Retry this channel* also retries.

Reason on the card	What it means	What you can do
Access denied — the stream is probably geo-restricted	The server refused your country (HTTP 403)	Nothing in the app; pick another channel
Stream not found — it's probably offline	The link is dead or the channel is off air (HTTP 404/410)	Try later, especially *Part-time* channels

Reason on the card	What it means	What you can do
This stream needs a login	The broadcaster requires an account (HTTP 401)	Not supported
The stream's server failed (HTTP 5xx)	The broadcaster's server has a problem	Retry later
Can't connect — the stream is offline or there's no internet	No route to the server	Check your internet, then retry
The stream timed out	The server did not answer in time	Retry; far servers are slower
Unsupported or broken stream format	The stream's format cannot be read	Pick another channel
This device can't decode the stream	The device lacks a decoder for it	Try another device
The stream ended	The broadcast stopped	Retry later

Extra notes the card can add: *The playlist marks it geo-blocked*, *The playlist marks it as not 24/7 — it may be off-air right now*.

Browser-only reasons

Browsers have security rules that the Android app does not. These channels usually play in the app.

Reason	Cause
Blocked by the browser (CORS) or the stream is offline	The stream's server does not allow web pages from other sites to read it
The browser blocks http:// streams on an https:// page	Mixed content: a secure page cannot load an insecure stream
It needs request headers a browser can't send	The channel requires a special User-Agent or Referer
The browser can't play this format, or the stream is offline	The browser lacks support for the format
It's a DASH stream, which this page can't play in a browser	The page has no DASH player in browsers; the app has one

Other status cards

Card	Meaning
Loading playlist…	The playlist is downloading
Tuning ‹name›…	Connecting to the channel
Buffering…	Waiting for more video from the server
Paused · press ▶❚❚ to resume	You paused the channel (Figure 1.8)

Banners

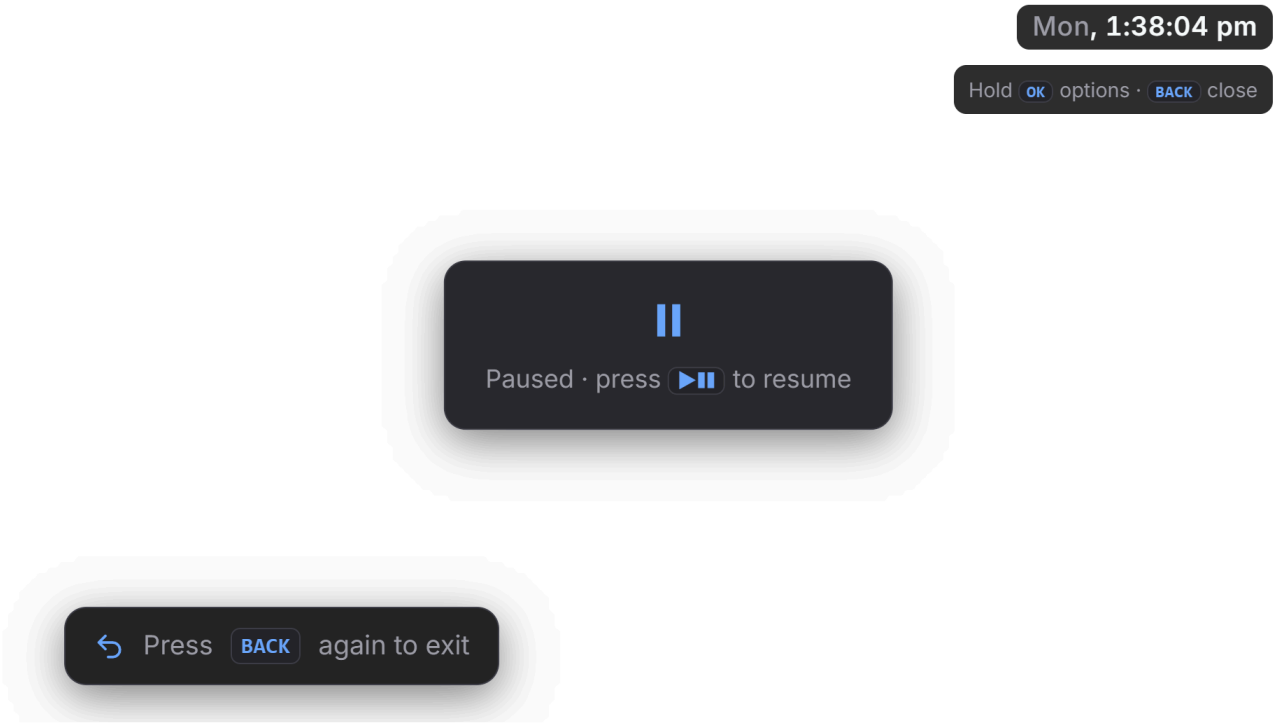


Figure 1.14 — A banner. This one appears after the first Back press with nothing open.

Banner	Meaning
11,302 channels from the default playlist	A playlist loaded
Playlist updated · 11,310 channels (+8)	A refresh found changes
Playlist load failed: …	The playlist could not be downloaded; the reason follows
Couldn't load your playlist (…) — showing the default one	Your playlist failed at start-up
No channels in ‹playlist›	The file has no channels in M3U format

Banner	Meaning
Could not read file	The chosen file could not be opened
Favourited ‹name› | Removed ‹name›	Star added or removed
Muted | Sound on	Sound toggled
Resumed muted — press any key or click to unmute	Browser only: the browser blocked sound on start; your next key, click or tap turns it on
No channel ‹number›	The typed number is past the end of the list
No previous channel yet	Nothing was watched before this channel
Previous channel isn't in the current filter	Clear the filter to reach it
That channel isn't in the current playlist	A recently watched channel was removed from the playlist
Nothing playing yet | Now watching: ‹name›	Answer to **Info**
Press BACK again to exit (*Go back again to exit* on touch)	A second Back within 2.5 seconds closes the app

1.13 Phones and tablets

On phones and tablets the app follows the way you hold the device.

	Portrait (upright)	Landscape (sideways)
Video	16:9 box at the top	Full screen
Channel list	Always visible below the video	Slides in from the left; open it with the **List** button
System bars	Status bar and navigation bar visible	Hidden; swipe from the screen edge to show them briefly
Clock	Hidden (the status bar shows the time)	Top right
Control bar	Over the video; tap the video to show it	Over the video; tap the video to show it

Fullscreen

Tap the **Expand** button on the control bar. In the Android app this turns the picture to landscape even if the phone is upright; tap **Shrink** or press **Back** to return. In a mobile browser it uses the browser's fullscreen mode and, where allowed, locks to landscape.

The list on touch screens

Touch screens have no 8-second auto-hide, because a slow finger-scroll is not counted as input. The list closes when you pick a channel, tap the video, or press **Back**.

Back button and gesture

The system Back button or swipe-back gesture works like the remote's **Back**, in this order:

1. Clears a channel number you are typing.
2. Closes the open menu, panel or picker.
3. Closes the channel list (landscape).
4. Leaves forced landscape (after **Expand**).
5. Shows *Go back again to exit*; a second Back within 2.5 seconds closes the app.

Typing

When you type a URL or a search, the on-screen keyboard pushes the page up so the field stays visible.

1.14 Privacy

The app has no accounts, no analytics and no advertising. Everything it remembers stays on your device.

Stored on your device

Data	Purpose
Favourite channels	*Favourites only* filter and stars
Last 15 channels watched (name, logo address, stream address, time)	Recently watched
Last channel	Resume on start
Filter settings	Restore your view
Playlist address, or the text of a small local playlist file	Load it at start

Data	Purpose
Whether you have seen the welcome card	Show it only once

This data is deleted when you uninstall the app, clear its storage in Android settings, or clear the page's site data in a browser.

Sent over the network

The app contacts only three kinds of server, all chosen by the playlist you load:

Server	Why	What it sees
Playlist server (iptv-org by default)	Download the channel list	Your IP address and the app's User-Agent (`MZIPTV/0.1 (Android TV)` in the app)
Stream servers	Play the channel you chose	Your IP address, the User-Agent, and any headers the playlist asks for
Logo servers	Show channel logos	Your IP address

The font, icons and video library are built into the app; nothing loads from third-party CDNs.

1.15 Troubleshooting and FAQ

Troubleshooting

Problem	Likely cause	Fix
Many channels show error cards	Normal for free channels: geo-blocks, dead links, part-time channels	Move on with ◀▶; use *Country* to pick channels from your region
Playlist load failed	No internet, wrong address, or the server is down	Check the connection and the URL; in a browser the server may also block other sites (CORS)
The list is empty with *No channels match these filters*	A filter hides everything	Press **OK** or tap **Clear filters**

Problem	Likely cause	Fix
Layout looks broken on a TV box	Android System WebView older than 62	Update *Android System WebView* from the Play Store (the app offers a link when the Play Store can update it; otherwise only a firmware update can)
Picture keeps buffering	Slow or far-away server, or a slow network	Try a channel from a nearer country, or a lower resolution
No sound after the app starts (browser)	The browser blocks autoplay with sound	Press any key, click or tap once
No sound (app)	Muted in the app, or TV volume	*Options* → *Unmute*, then check the TV volume
Remote button does nothing	Some remotes lack MENU, colour or CH± buttons	Hold **OK** for the options menu: favourite, recent, filter, mute, previous channel, refresh
Back exits too easily, or not at all	Exit needs two Backs within 2.5 s	Press Back once to close panels; twice quickly to exit
Update will not install ("package conflicts")	The new APK has a different signature	Uninstall the old app first (favourites are lost)
Your playlist disappeared after restart (browser)	The file was larger than about 1.5 MB	Open the file again, or host it and load it by URL
A channel plays in the app but not in the browser	Browser security rules (CORS, mixed content, headers)	Use the Android app for that channel
A channel never plays, often a DASH channel from a paid service	It is copy-protected (DRM)	Not supported; pick another channel

Frequently asked questions

Is the app free? Yes. It is open-source software under the MIT licence.

Where do the channels come from? From the playlist you load. The default is the community-maintained iptv-org list of publicly available streams. The app itself supplies no streams.

Can I record or rewind? No. The app plays live streams at the live moment; pausing and resuming returns to live.

Is there a programme guide (EPG)? Not yet.

Can I use a VPN to watch geo-blocked channels? The app does not include or recommend ways around restrictions. Geo-blocked channels show their error card.

Will my favourites survive an update? Yes, when you install the new APK over the old one. They are lost only if you uninstall first.

Can I sort or renumber channels? Channels keep the playlist's order. Make your own playlist (1.11) to choose the order, or use filters and favourites.

Does the app work offline? No. It needs the internet for the playlist and the streams.

Which remote buttons are required? The D-pad, **OK** and **Back** are enough to browse, play and change channels. Hold **OK** for the options menu, which covers favourites, recently watched, filter, mute and refresh. Pausing needs a Play/Pause button.

How do I report a problem? Open an issue on the GitHub repository with your device, Android version, and the message you saw.

Part 2. Architecture Overview

The product is one web page with two hosts. In a browser, the page plays video itself with `<video>` and hls.js. In the Android app, a transparent WebView shows the page on top of a native ExoPlayer view, and a small JavaScript bridge connects them.

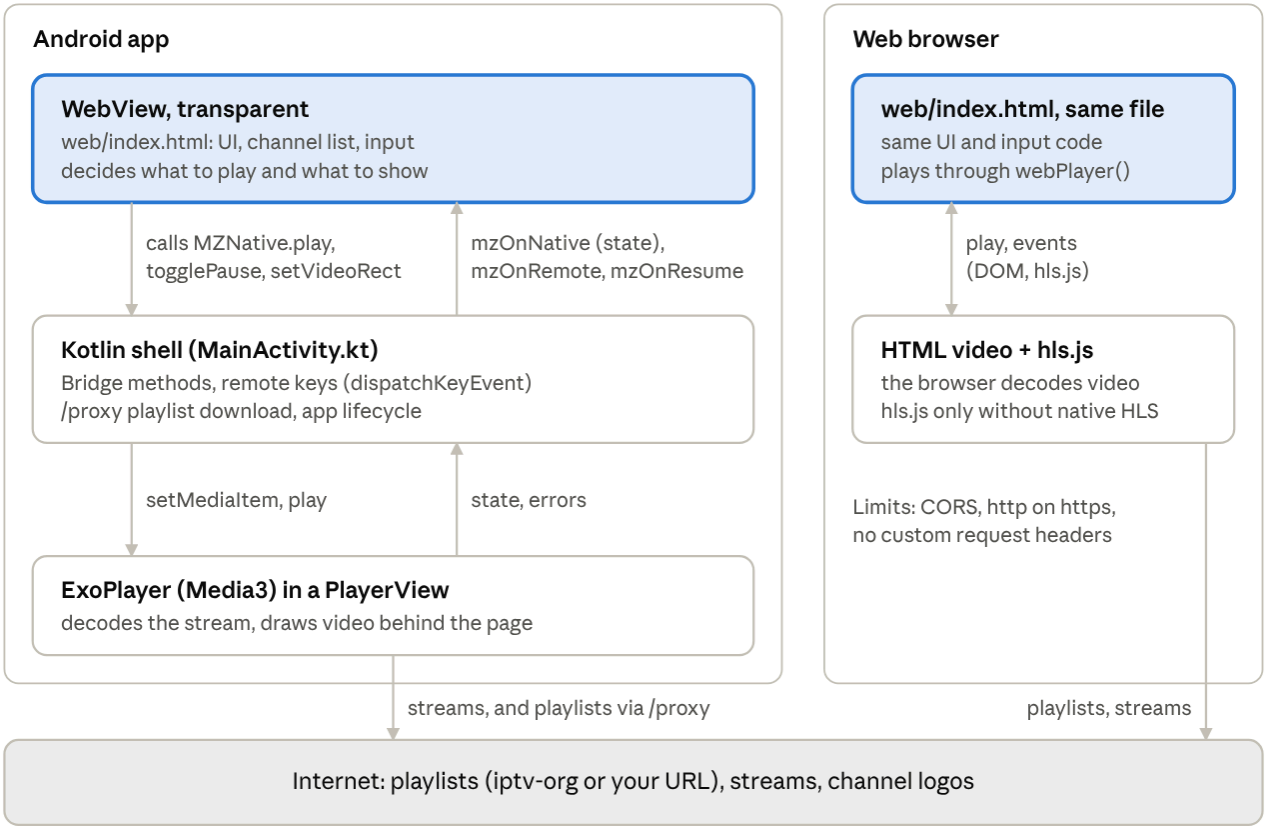
2.1 Design goals

Goal	How the design meets it
One interface for TV, phone and browser	`web/` is copied into the APK as assets at build time
Native-quality playback on Android	ExoPlayer (Media3 1.5.1) decodes video; the page never touches media in the app
Works on cheap TV boxes	No framework, list virtualization, hex colours instead of `oklch()`, fallbacks for WebViews back to 62 (Android 9 ships 66)
No third-party servers	Inter font and hls.js 1.5.13 are bundled in `web/vendor/`; only playlists, streams and logos use the network

Goal	How the design meets it
Honest failures	Every error is translated into a plain sentence; restrictions are reported, never bypassed

2.2 System context

One page, two hosts: in the app, native video plays under the page



system context · app and browser hosts

The same `web/index.html` runs in both hosts. In the app it only describes what to play; Kotlin and ExoPlayer do the network and decoding work beneath it.

2.3 Repository layout

Path	Contents	Language
`web/index.html`	The whole interface: HTML, CSS and JS in one file (about 2,150 lines)	HTML, CSS, JavaScript

Path	Contents	Language
`web/vendor/`	hls.js 1.5.13 and the Inter variable font, with licences	—
`web/icons/`	Browser tab and home-screen icons (generated)	PNG
`android/app/src/main/java/com/mazem3dia/iptv/MainActivity.kt`	The native shell: WebView, ExoPlayer, bridge, key handling, playlist proxy (about 600 lines)	Kotlin
`android/app/build.gradle.kts`	Build config; copies `../../web` into the APK assets	Kotlin DSL
`android/app/src/main/AndroidManifest.xml`	Permissions, TV launcher banner, orientation handling	XML
`tests/`	Eight suites that drive the page in headless Chrome, plus Firefox and WebKit through Playwright	JavaScript (Node ESM)
`package.json`	Dev tooling only: test scripts and Playwright. The player itself has no npm dependencies	JSON
`art/`	Icon source and `make-icons.py`, which generates every icon and store graphic	Python, HTML

2.4 Runtime flow: launch to first picture

1. **Host detection.** An inline script in `<head>` sets the `<html>` class to `is-native` or `is-web` before first paint. Later, the constant `NATIVE` records whether the bridge object `MZNative` exists.
2. **Restore state.** `loadStore()` reads favourites, history, filters and the saved playlist source from `localStorage`.
3. **Download the playlist.** `loadUrl()` fetches the saved URL or the default. In the app the request goes to `/proxy?url=…`, which Kotlin downloads natively.
4. **Parse.** `parseM3U()` turns the text into channel objects stored in `channels`.
5. **Filter.** `applyFilters()` builds `view`, the subset the list shows.
6. **Render.** `renderWindow()` draws only the rows inside the viewport.
7. **Resume.** If the last channel is in `view`, `playIndex()` tunes it automatically.
8. **Play.** `player.play(ch)` either calls `MZNative.play(…)` (app) or sets up `<video>`/hls.js (browser).
9. **Report.** The player reports `buffering`, `playing`, `paused` or an error back to `onPlayerState()` / `onPlayerError()`, which update the status card.

Part 3. Code Fundamentals

Ten ideas explain almost all of `web/index.html`. Learn them in this order; each builds on the one before.

3.1 Host detection: one flag decides

The page checks once whether it runs inside the app, and every platform difference reads that one flag.

```
// index.html:613
const NATIVE = !!(window.MZNative && typeof MZNative.play === "function");
```

- `MZNative` exists only when Kotlin injects it with `addJavascriptInterface`.
- A second flag, `TOUCH`, is true on touch screens that are not a TV. `useButtons()` combines both to choose between on-screen buttons and keyboard hints.
- CSS reacts to the `is-native` / `is-web` class on `<html>`, set before first paint so there is no flicker.

Principle: detect a capability once, store it in a constant, branch on the constant. Never scatter `if (window.MZNative)` checks.

3.2 The state model

All UI state lives in a handful of top-level variables (`index.html:645–654`). Reading this block first gives you the map of the app.

Variable	Holds
`channels`	Every channel from the playlist: the single source of truth
`view`	The filtered subset currently shown; always rebuilt from `channels`
`focusIndex`	Which row of `view` has focus
`playingCh` , `playingUrl` , `prevUrl`	What is playing and what played before
`playState`	`idle` , `loading` , `buffering` , `playing` , `paused` or `error`
`mode`	Which screen owns input: `list` , `filter` , `welcome` , `recent` or `menu`
`zone`	Inside list mode: the channel rows or the top bar (`head`)
`filter`	`{ search, favOnly, country, category }`
`favorites` , `recentlyWatched` , `lastChannelKey`	Persisted user data

Principle: keep state in few, named places and derive everything else from it. `view` is never edited directly; it is recomputed.

3.3 The data pipeline

Data flows one way, from network text to DOM rows:

```
loadUrl(url)          fetch the playlist text
  → ingest(text)      replace the master list
    → parseM3U(text)  text → channel objects
    → channels[]      master list
    → applyFilters()  channels + filter → view[]
    → relayoutList()  size the scroll area
    → renderWindow()  build only the visible rows
```

Any change of input (new playlist, new filter, new favourite) re-enters the pipeline at the right step instead of patching the DOM by hand.

3.4 Parsing M3U

An M3U playlist is plain text. Each channel is an `#EXTINF` line with attributes and a name, optional `#EXTVLCOPT` header lines, then the stream URL.

```
#EXTINF:-1 tvg-id="NHKWorld.jp" tvg-logo="https://…/nhk.png" group-
title="News",NHK World (1080p)
#EXTVLCOPT:http-user-agent=Mozilla/5.0
https://example.com/nhk/index.m3u8
```

`parseM3U()` (`index.html:790`) reads line by line with one variable, `cur`, holding the channel being built:

1. `#EXTINF` starts a new `cur`: attributes are pulled out with a small regex helper, `attr(line, name)`.
2. `parseName()` strips tags from the name and turns them into fields: `(1080p)` → `res: 1080`, `[Geo-blocked]` → `geo: true`, `[Not 24/7]` → `part: true`.
3. The country comes from the `tvg-id` suffix (`NHKWorld.jp` → `JP`).
4. A precomputed lowercase `find` string (name + country name + genres) makes search a single `includes()` call.
5. The first non-comment line after `#EXTINF` is the URL: it closes `cur` and pushes it to the output.
6. `key` is set to `"id:" + tvg-id` when present, else the URL. Favourites and history use `key`, because stream URLs change more often than IDs.

Principle: do expensive work once at parse time (the `find` string, the country) so per-keystroke work stays cheap.

3.5 List virtualization

The default playlist has more than 11,000 channels. Creating 11,000 DOM rows would freeze a TV box, so the list draws only what is on screen plus a small buffer.

```
// index.html:942
const first = Math.max(0, Math.floor(scrollTop / ROW_H) - BUFFER);
const last  = Math.min(view.length - 1, Math.ceil((scrollTop + viewportH) /
ROW_H) + BUFFER);
```

- A tall empty div, `sizer`, gets `height = view.length × ROW_H`, so the scrollbar behaves as if every row existed.
- Each row is absolutely positioned at `i × ROW_H`.
- `rendered` is a `Map` from index to element. Rows outside `[first, last]` are removed; missing rows are built by `buildRow(i)`.

- `ROW_H` is measured from the root font size in `measure()`, so it stays correct on every screen size.
- Scroll events are throttled to one `renderWindow()` per animation frame with `requestAnimationFrame`.

Principle: render cost should depend on the screen size, not the data size.

3.6 The player interface (strategy pattern)

Two objects implement the same three methods. The rest of the code calls `player` and never asks which one it got.

```
// index.html:1154
const player = NATIVE ? nativePlayer() : webPlayer();

// both provide:
player.play(ch, opts)   // start a channel
player.togglePause()    // pause / resume at the live edge
player.setMuted(bool)
```

	`nativePlayer()`	`webPlayer()`
Decoder	ExoPlayer, in Kotlin	Browser `<video>`; hls.js when the browser lacks native HLS
How it plays	Calls `MZNative.play(url, isHls, ua, referrer)`	Sets `video.src` or attaches hls.js
How it reports	Kotlin calls `window.mzOnNative({type, …})`	`<video>` events and hls.js errors
Common sink	`onPlayerState(s)` and `onPlayerError(reason)`	Same

hls.js (about 400 KB) is loaded lazily, only when a browser actually needs it.

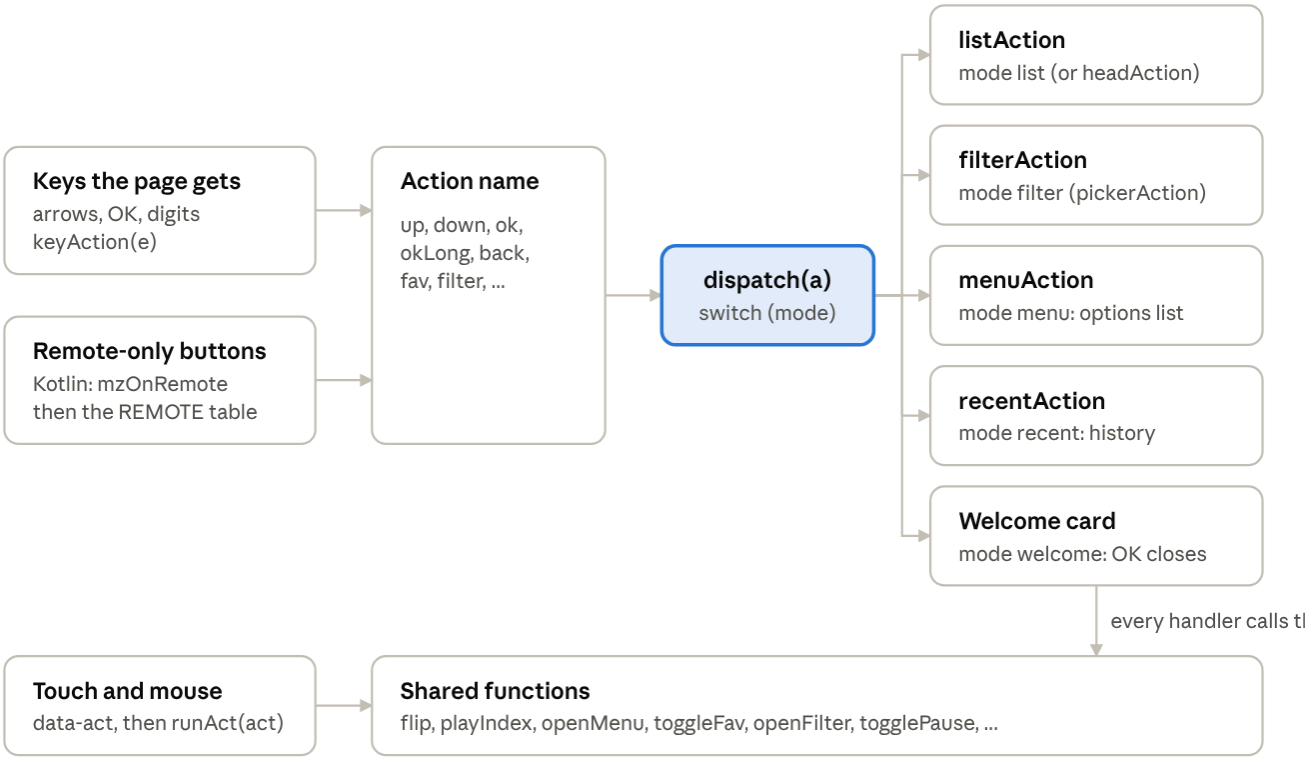
Principle: hide platform differences behind a small interface. Adding a third player (say, a desktop wrapper) would touch only this block.

3.7 Input: keys become actions, actions go to the current mode

Input is handled in three steps, so the same logic serves keyboards, TV remotes and touch.

1. **Translate.** `keyAction(e)` maps raw keys to action names: `ArrowUp` → `up`, `PageDown` → `chPlus`, `f` → `fav`. Remote buttons the WebView never receives arrive from Kotlin as names (`"Menu"`, `"Red"`) and are mapped through the `REMOTE` table.
2. **Route.** `dispatch(action)` looks at `mode` and calls one handler: `listAction`, `filterAction`, `pickerAction`, `menuAction`, `recentAction`, or `headAction` when the top bar has focus.
3. **Act.** The handler decides what the action means in that mode. `ok` plays a channel in the list, toggles an option in the filter panel, and runs an item in the menu.

Every input becomes one named action, routed by the current mode



input path · 3 sources, 1 dispatcher, 5 mode handlers

Keys and remote buttons meet at one action name; `dispatch()` hands it to the handler of the current mode. Touch skips the action step and calls the same functions.

Touch and mouse use the same functions directly: buttons carry a `data-act` attribute, and one click listener calls `runAct(act)`.

Principle: this is a small state machine. Adding a screen means adding a mode and a handler; existing handlers do not change.

3.8 Persistence

Everything the user expects to survive a restart is one JSON object in `localStorage` under `tv.iptv.store.v1`.

```
// index.html:703
localStorage.setItem(STORE_KEY, JSON.stringify({
  source: savedSource, lastChannelKey, favorites: [...favorites],
  filter, seenWelcome, recentlyWatched,
}));
```

- Both `loadStore()` and `saveStore()` are wrapped in `try/catch`: private windows and full storage must not break the page.
- A `Set` is stored as an array and rebuilt on load.
- The `v1` suffix leaves room for a future schema change.
- The playlist itself is not cached: it downloads fresh each session.

3.9 Escaping untrusted text

Playlist text comes from the internet, and the page builds HTML strings for `innerHTML`. Every value from a playlist passes through `esc()` first.

```
// index.html:657
const ESC = { "&":"&amp;", "<":"&lt;", ">":"&gt;", '"':"&quot;", "'":"&#39;"
};
const esc = (s) => String(s == null ? "" : s).replace(/[&<>"']/g, (c) =>
ESC[c]);
```

Without it, a channel named `<img src=x onerror=…>` would run script inside the app, which has a bridge to native code. Where possible the code uses `textContent`, which never parses HTML.

Rule: any string that did not come from this file goes through `esc()` or `textContent`.

3.10 Styling: tokens and rem

- **Design tokens.** Colours are CSS variables on `:root` (`--background`, `--primary`, `--muted-foreground` and so on). Components use the variables, never raw colours. The palette is shadcn zinc with a blue primary taken from the app icon.
- **Contrast.** Every text colour is documented with its WCAG ratio; the `contrast` test suite enforces at least 4.5:1 over dark and bright video.
- **Hex, not `oklch()`.** `oklch()` needs WebView 111+, and many TV boxes ship older WebViews.

- **rem sizing.** The root font size follows the screen's short side: a 1080p TV (960×540 CSS px) gets 18 px, a phone 15 px. Everything else is in `rem`, so one stylesheet scales to every device.
- **TV safe area.** `--safe-x` and `--safe-y` keep content about 5% from the edges, because many TVs crop the picture (overscan).
- **Icons** are an inline SVG sprite (`<symbol id="i-play">` …) used through `<use href="#i-play">`, so they render identically on every TV, unlike emoji.

Part 4. Advanced Techniques

These techniques solve the hard edges: two languages talking across threads, async races, browser security rules, TV remotes, and device quirks. Each section states the problem, the solution, and where it lives.

4.1 The JavaScript ↔ Kotlin bridge

Problem. The page decides what to play; Kotlin owns the player. They run in different languages and on different threads.

Solution. Two one-way channels, both carrying simple values (strings, booleans, numbers, JSON).

Direction	Mechanism	Example
Page → native	`addJavascriptInterface(Bridge(), "MZNative")`; methods marked `@JavascriptInterface` become callable from JS	`MZNative.play(url, isHls, ua, referrer)`
Native → page	`webView.evaluateJavascript("…")` runs a JS expression in the page	`mzOnNative({"type":"state","state":"playing"})`

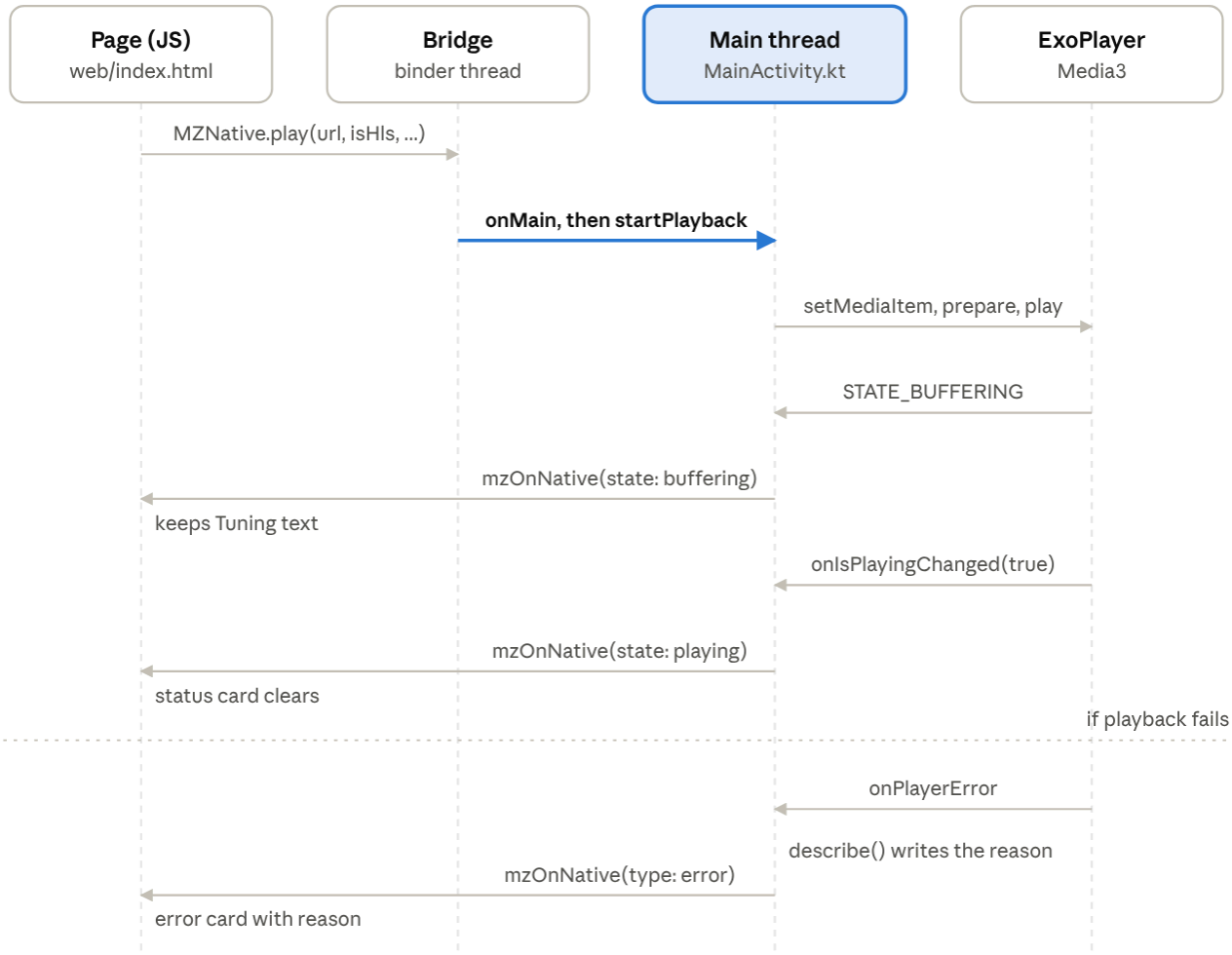
Threading. Bridge methods run on a WebView binder thread, but ExoPlayer and views may only be touched on the main thread. Every bridge call hops over with `onMain {}`, which also drops calls that arrive while the activity is closing:

```
// MainActivity.kt:225
private fun onMain(block: () -> Unit) {
    main.post { if (!isFinishing && !isDestroyed) block() }
}
```

Defensive calls. Native → page calls are guarded, e.g. `window.mzOnNative && mzOnNative(…)`, so an early event before the page has loaded does nothing instead of throwing.

Security surface. Only the methods listed in `Bridge` are exposed. The WebView never navigates away from its own asset host (`shouldOverrideUrlLoading`), file and content access are off, and all playlist text is escaped (3.9). Together these stop a hostile playlist from reaching the bridge.

A tune request hops to the main thread; state flows back as events



tuning a channel in the app · call sequence

The highlighted hop is the one rule to remember: bridge calls arrive off the main thread, so every one is posted to the main thread before it touches ExoPlayer.

4.2 "Latest wins": cancelling stale async work

Problem. A user flicks through five channels in two seconds. Each `play()` starts async work (loading hls.js, waiting for the manifest). If an older request finishes last, the wrong channel plays.

Solution. A counter that every new request increments; each async step checks it is still the newest.

```
// webPlayer, index.html:1115
async play(ch, opts) {
  const my = ++token;           // this request's ticket
  teardown();
  await loadHlsJs();
  if (my !== token) return;     // a newer play() started: give up quietly
  …
}
```

The same pattern guards playlist downloads (`loadSeq` in `loadUrl()`). It is simpler than `AbortController` and works for any promise, not only `fetch`.

4.3 Downloading playlists without CORS: the native proxy

Problem. Browsers block cross-origin `fetch()` unless the server sends CORS headers, and most IPTV playlist servers do not.

Solution. In the app, the page is served from `https://appassets.androidplatform.net/assets/` by `WebViewAssetLoader`. The page fetches `/proxy?url=<playlist>` on that same origin, so there is no cross-origin request. Kotlin intercepts it in `shouldInterceptRequest` and downloads the real URL with `HttpURLConnection` (`MainActivity.kt:405`).

- Redirects are followed manually (up to 5 hops), because Java will not follow a redirect between `http` and `https`.
- Errors come back as plain-text HTTP 400 or 502 responses, which `loadUrl()` shows to the user.
- The response is marked `Cache-Control: no-store`, so every session gets a fresh playlist.

4.4 Placing native video under a web layout

Problem. In portrait on a phone, the video must sit in a 16:9 box at the top of the page, but the video is a native view the page cannot style.

Solution. The page measures its own `<video>` element and tells Kotlin where to put the native view.

```
// index.html:1928
function syncVideoRect() {
  if (!NATIVE) return;
  if (!PORTRAIT.matches) { MZNative.setVideoRect(0, 0, -1, -1); return; }  //
whole screen
  const r = els.video.getBoundingClientRect();
  MZNative.setVideoRect(r.left, r.top, r.width, r.height);
}
```

In a WebView that fills the window, one CSS pixel equals one Android dp, so Kotlin only multiplies by the display density (`placeVideo()`, `MainActivity.kt:283`). The page calls `syncVideoRect()` on start, resize and orientation change.

4.5 Remote keys and long-press OK

Problem 1. The WebView receives D-pad, OK and digits as DOM key events, but not Back, Menu, CH±, media or colour buttons.

Solution. `dispatchKeyEvent()` in Kotlin catches those keys, looks them up in `REMOTE_KEYS`, and calls `mzOnRemote("<name>")`. The page maps the name through `REMOTE` to an action and returns `true` if it used the key. For Back, a `false` result makes Kotlin call `finish()`, so the page alone decides when Back exits.

Problem 2. "Hold OK for options" needs to tell a tap from a hold, but Android WebView does not set `e.repeat` on held keys.

Solution. The page tracks held keys itself in a `Set`:

1. First `keydown` of Enter: arm a 550 ms timer (`LONG_PRESS`).
2. A second `keydown` before `keyup` is a repeat, meaning the system detected a hold: fire the long press at once.
3. Timer expires: fire the long press.
4. `keyup` before either: it was a tap, so dispatch `ok`.
5. Window `blur` clears everything, so a key held while switching apps cannot get stuck.

Kotlin also rewrites game-controller A, Select and keypad Enter to `DPAD_CENTER` before the WebView sees them, so long-press works on those too.

4.6 Live TV lifecycle

Problem. Live TV cannot meaningfully pause in the background: when the user returns, they expect the live picture, not a frame from ten minutes ago.

Solution.

- `onStop()` remembers whether it was playing, then calls `player.stop()` to release the network and decoder.
- `onStart()` calls `seekToDefaultPosition()` (the live edge), `prepare()` and `play()`.
- Pause/resume works the same way: resume seeks to the live edge in both players.
- `ERROR_CODE_BEHIND_LIVE_WINDOW` (fallen too far behind) is recovered silently by re-seeking to the live edge.
- `keepScreenOn` follows `isPlaying`, so the screensaver never starts mid-programme.

4.7 Background playlist refresh

The iptv-org playlist changes daily, and TV apps stay open for days. `maybeRefresh()` re-downloads it when it is older than 6 hours (`REFRESH_MS`).

- **When it checks:** every 15 minutes, when the browser tab becomes visible, and when Kotlin calls `mzOnResume()` from `onStart()`.
- **When it waits:** while a menu is open, while digits are being typed, or within 60 seconds of input in an open list, because rows would move under the user.
- **What it keeps:** `refreshInPlace()` re-points `playingCh` at the same channel in the new list (by URL, else by key), so playback never stops.

4.8 Autoplay with sound in browsers

Problem. Browsers refuse `video.play()` with sound until the user has interacted with the page, which breaks "resume last channel on start".

Solution. On an auto-resume, a `NotAllowedError` triggers `fallbackToMuted()`: play muted and show "Resumed muted — press any key or click to unmute". The next key, click or remote press calls `clearPendingUnmute()`, which turns the sound on. The app is not affected: it sets `mediaPlaybackRequiresUserGesture = false`.

4.9 Turning failures into sentences

Both layers map technical errors to the same plain messages, so the error card reads the same in the browser and the app.

Source	Browser (`webPlayer`)	App (`describe()` in Kotlin)
HTTP status	`httpReason(code)` from hls.js response	`InvalidResponseCodeException.responseCode` found by walking the cause chain
Network	hls.js `networkError` → CORS or offline	`ERROR_CODE_IO_NETWORK_CONNECTION_FAILED` / `_TIMEOUT`

Source	Browser (`webPlayer`)	App (`describe()` in Kotlin)
Format	`MediaError` codes 3 and 4	Error code ranges 3000–3999 (parsing) and 4000–4999 (decoding)
Playlist hints	`[Geo-blocked]`, `[Not 24/7]`, required headers	Same notes, added by the page

Kotlin also retries once as HLS when a stream fails with `ERROR_CODE_PARSING_CONTAINER_UNSUPPORTED`, because some HLS links do not end in `.m3u8`.

DASH (`.mpd`) streams play through Media3's DASH module: with no MIME type set, `DefaultMediaSourceFactory` picks it from the URL. There is no DRM support, so copy-protected streams fail.

4.10 Per-channel request headers

Some streams refuse requests without a specific User-Agent or Referer. The parser reads them from `http-user-agent`, `http-referrer` and `#EXTVLCOPT` lines. In the app, `startPlayback()` sets them on the `DefaultHttpDataSource.Factory` before each channel, and resets them for channels that need none. Browsers forbid setting these headers, so the page tells the user such a channel should play in the app.

4.11 Data migration

Older builds keyed favourites and history by stream URL. `migrateKeys()` runs after each playlist load: when it sees a channel whose old URL key is still stored, it swaps in the stable `id:<tvg-id>` key and saves. Users keep their favourites across the upgrade without a version check.

4.12 Device quirks

- **Emulator decoder bug.** The emulator's `c2.goldfish` decoder claims it can switch resolution in place but keeps the old size, so a new channel was drawn stretched or cropped. `PlayerRenderers` overrides `canReuseCodec()` to force a fresh decoder only when the codec name contains `goldfish`; real devices keep seamless reuse.
- **Old WebViews.** TV boxes keep the WebView they shipped with: 66 on stock Android 9. The page has a fallback for each feature that version lacks, and newer browsers never match them:
  - Flex `gap` (Chrome 84): a head script sets `html.no-gap`, and margin rules take over.

  - `min()`, `max()`, `clamp()` (79): `legacySizes()` in `measure()` works out the root size and `--safe-x`, `--safe-y`, `--strip-w`; an `@supports not (width: min(1px, 2px))` block covers the card widths.
  - `Intl.DisplayNames` (81): country names come from the bundled `REGION_EN` table.
  - Rounded outlines (94): focus rings are drawn as shadows.
  - Below WebView 62, `checkWebViewVersion()` shows a dialog once per WebView version. *Update* appears only for Play Store providers (`com.google.android.webview`, Chrome).
- **System bars.** Portrait phones keep the status and navigation bars; landscape and TV hide them. Window insets pad the root view, so neither the page nor the video sits under a bar, notch or on-screen keyboard.

Part 5. Developer Guide

UI changes are made in `web/index.html` and checked in a browser first; the Android build picks them up on the next rebuild.

5.1 Prerequisites

Tool	Version	Used for
Android Studio	Current	Android SDK, emulator, Logcat
JDK for Gradle	17–23 (JDK 21 recommended)	Gradle 8.11.1 does not run on Java 25, which newer Android Studio bundles
Node.js	22+	Test runner
Google Chrome	Current	Headless browser for tests (Microsoft Edge works too); `chrome://inspect` for debugging
Python + Pillow + numpy	3.x	Only for regenerating icons (`art/make-icons.py`)
Playwright (optional)	1.64.1	Firefox and WebKit for the cross-browser suite

Build stack: Android Gradle Plugin 8.7.3, Kotlin 2.0.21, Media3 1.5.1 (ExoPlayer, HLS, DASH), `compileSdk` / `targetSdk` 35, `minSdk` 28 (Android 9).

5.2 Building the APK

1. In Android Studio, **File → Open** the `android/` folder (not the repository root).
2. If asked to select a Gradle JVM, choose **JVM 21**.
3. Wait for Gradle sync, pick a device, press **Run**.

From the command line:

```
cd android
./gradlew assembleDebug      # Windows: gradlew.bat assembleDebug
./gradlew assembleRelease
```

APKs are named `MZ-IPTV-Player-<version>-<debug|release>.apk` under `android/app/build/outputs/apk/`.

5.3 Release signing

Android installs an update only if it is signed with the same key as the installed app. A lost key means existing users can never update.

1. Create the key once and store it outside the repository, with a backup.
2. Copy `android/keystore.properties.example` to `android/keystore.properties` (git-ignored) and fill in the path, alias and passwords.
3. Bump `versionCode` (must increase every release) and `versionName` in `android/app/build.gradle.kts`.
4. Run `./gradlew assembleRelease`.

Without `keystore.properties`, release builds fall back to the debug key: they install, but cannot update an install signed with the real key.

5.4 Running the tests

The suites drive `web/index.html` in headless Chrome (the cross-browser suite: Firefox and WebKit through Playwright); app behaviour is tested with a simulated `MZNative` bridge.

```
node tests/run.mjs            # 6 offline suites, about 2 minutes
node tests/run.mjs --online   # + real playlist and real playback
node tests/run.mjs contrast   # only suites whose name matches
```

Suite	Checks
`keys-remote`	Keyboard and remote: browsing, OK and hold, Back, digits, filters, favourites, refresh, default playlist
`app-devices`	App on phone, tablet and TV: touch UI, video box, Back, rotation
`screens`	5 screen sizes with touch; no horizontal overflow
`network`	Nothing loads from a CDN; font and icons are local
`contrast`	WCAG text contrast (4.5:1, or 3:1 for large text) and focus outlines (3:1) over black and white video
`cross-browser`	Firefox and WebKit at TV, desktop and phone sizes: layout, keyboard and touch, search, menus, cards; with --online also real HLS playback. Optional: skipped without Playwright
`online-playlist`	The real iptv-org playlist parses; search works
`online-playback`	The default playlist downloads and a real channel plays

To enable the cross-browser suite, run `npm install`, then `npx playwright install firefox webkit` once (it downloads Firefox and WebKit, a few hundred MB). WebKit builds without media support, such as Playwright's Windows build, skip the playback check; real Safari plays HLS natively.

Screenshots and logs go to `tests/out/<suite>/` (git-ignored). Run the offline suites after every change.

5.5 Debugging

- **Page:** with a debug build running, open `chrome://inspect` on the computer to get DevTools for the WebView. `console.log` from the page also appears in Logcat under tag `MZIPTV`.
- **Native:** Logcat filter `package:com.mazem3dia.iptv`. Tag `MZIPTV-player` logs every player event (formats, sizes, dropped frames) in debug builds.
- **Browser:** open `web/index.html` directly and use normal DevTools.

5.6 Coding conventions

- **No framework, no build step for the page.** Plain DOM APIs and one HTML file keep the page fast on TV boxes and readable without tooling.
- **No CDNs.** Any new library goes into `web/vendor/` with its licence. The `network` suite fails otherwise.
- **Comments explain why,** not what: a reason, a device quirk, or a source. Match that density.
- **Escape** every external string (3.9).
- **CSS** uses the existing tokens and `rem`; check new text with the `contrast` suite.
- **Kotlin** keeps the shell thin: decide behaviour in the page where possible, and expose only small, value-typed bridge methods.
- **Line endings** are LF; vendor files are kept byte-exact (`.gitattributes`).

5.7 How to extend

Add a remote button

1. **Kotlin:** add `KeyEvent.KEYCODE_X to "Name"` to `REMOTE_KEYS` in `MainActivity.kt`. Add it to `REPEATABLE` if holding it should repeat.
2. **Page:** add `Name: "action"` to the `REMOTE` table.
3. **Page:** handle `"action"` in `listAction()` (and other mode handlers if it applies there).
4. **Keyboard equivalent:** add a key to `keyAction()` so the browser and tests can reach it.
5. Update the controls guide in `applyPlatformCopy()` and add a case to `tests/keys-remote.mjs`.

Add a screen (mode)

1. Add the markup and styles.
2. Write `openX()` / `closeX()` that set `mode = "x"` and back to `"list"`.
3. Write `xAction(a)` returning `true` for every action it consumes (Back must close it).
4. Add `case "x": return xAction(a);` to `dispatch()`.
5. Make sure `maybeRefresh()` waits while the screen is open (it already waits for any mode other than `list`).

Add a bridge call

1. **Kotlin:** add a method to `Bridge` with `@JavascriptInterface`, taking and returning only primitives or strings. Wrap any player or view work in `onMain {}`.

2. **Page:** call it only when `NATIVE` is true, and add a no-op or browser equivalent if the feature also makes sense there.
3. **Tests:** add the method to the simulated `MZNative` in the test suites.
4. Document it in Part 6.1.

5.8 Change checklist

- [ ] Change works in a desktop browser with keyboard and mouse
- [ ] Change works in a phone-size browser window with touch emulation
- [ ] `node tests/run.mjs` passes
- [ ] New external strings go through `esc()`
- [ ] No new network origin besides playlists, streams and logos
- [ ] Android build runs on an emulator or device if Kotlin or the bridge changed
- [ ] `versionCode` bumped for a release

Part 6. Reference

6.1 Bridge API: page → native (`window.MZNative`)

Available only in the Android app. All calls are asynchronous except `isTv()`.

Method	Parameters	Effect
`play(url, isHls, userAgent, referrer)`	`string, boolean, string, string`; empty string = none	Sets request headers, loads the stream, starts playback
`togglePause()`	—	Pause; or resume at the live edge
`setMuted(muted)`	`boolean`	Volume 0 or 1
`setVideoRect(x, y, w, h)`	CSS px; `w` or `h` ⩽ 0 = whole screen	Moves and sizes the native video view
`setLandscape(on)`	`boolean`	Forces sensor landscape, or releases orientation
`isTv()`	— → `boolean`	True when the UI mode is television
`exit()`	—	Closes the app

Same-origin HTTP endpoint: `GET /proxy?url=<http(s) URL>` returns the playlist as `text/plain`, or 400/502 with a readable message.

6.2 Bridge API: native → page

Function	Argument	When Kotlin calls it
`mzOnNative(ev)`	`{type:"state", state:"buffering"|"playing"|"paused"}` or `{type:"error", reason}`	Player state changes and errors
`mzOnRemote(name)`	Remote button name (6.3); returns `true` if consumed	A remote button the WebView does not receive
`mzOnResume()`	—	Activity `onStart()`: app is back on screen

6.3 Remote key map

Android key code	Name sent	Page action
`BACK`, `BUTTON_B`	`Back`	`back`
`MENU`	`Menu`	`options`
`SEARCH`	`Search`	`search`
`GUIDE`	`Guide`	`guide`
`INFO`	`Info`	`info`
`CHANNEL_UP` / `CHANNEL_DOWN`	`ChannelUp` / `ChannelDown`	`chPlus` / `chMinus` (repeat when held)
`LAST_CHANNEL`	`LastChannel`	`last`
`MEDIA_PLAY_PAUSE`, `BUTTON_START`	`PlayPause`	`playPause`
`MEDIA_PLAY`	`Play`	`play`
`MEDIA_PAUSE`, `MEDIA_STOP`	`Pause`	`pause`
`MEDIA_NEXT`, `MEDIA_FAST_FORWARD`, `BUTTON_R1`	`Next`	`right` (next channel)

Android key code	Name sent	Page action
`MEDIA_PREVIOUS`, `MEDIA_REWIND`, `BUTTON_L1`	`Previous`	`left` (previous channel)
`BOOKMARK`	`Bookmark`	`fav`
`PROG_RED` / `GREEN` / `YELLOW` / `BLUE`	`Red` / `Green` / `Yellow` / `Blue`	`fav` / `recent` / `filter` / `last`
`BUTTON_A`, `BUTTON_SELECT`, `NUMPAD_ENTER`	rewritten to `DPAD_CENTER`	`ok` (via DOM Enter)

6.4 Keyboard map (`keyAction()`)

`e.key`	Action
`ArrowUp` / `ArrowDown` / `ArrowLeft` / `ArrowRight`	`up` / `down` / `left` / `right`
`Enter`	`ok`, or `okLong` when held 550 ms
`Escape`, `GoBack`, `BrowserBack`	`back`
`Backspace`	`erase`
`PageDown`, `ChannelUp` / `PageUp`, `ChannelDown`	`chPlus` / `chMinus`
`0`–`9`	`digit`
`/`	`filter`
`ContextMenu`	`options`
Space, `MediaPlayPause`	`playPause`
`Info`, `i`	`info`
`f` / `r` / `m`	`fav` / `recent` / `mute`

Keys with Ctrl, Alt or Cmd are left to the browser.

6.5 Channel object (output of `parseM3U()`)

Field	Type	Source / meaning
`name`	string	Display name with tags removed
`url`	string	Stream URL

Field	Type	Source / meaning
`key`	string	`id:<tvg-id>`, or the URL when there is no ID; identity for favourites and history
`tvg`	string	`tvg-id` attribute
`logo`	string	`tvg-logo` attribute
`groups`	string[]	`group-title` split on `;` ; `["Uncategorized"]` if empty
`country`	string | null	Upper-case code from the `tvg-id` suffix
`feed`	string	Part after `@` in `tvg-id`, e.g. `HD`
`res`	number	Vertical resolution from `(1080p)`; 0 if unknown
`geo`	boolean	Name contained `[Geo-blocked]`
`part`	boolean	Name contained `[Not 24/7]`
`isHls`	boolean	URL contains `.m3u8`
`ua`, `ref`	string	Required User-Agent and Referer, if any
`find`	string	Lower-case search text: name, country name, genres

6.6 Storage schema (`localStorage["tv.iptv.store.v1"]`)

Key	Type	Meaning
`source`	`{type:"url", url}` | `{type:"text", name, text}` | `{type:"file", name}` | null	Playlist to load at start; local files under 1.5 MB are stored as text
`lastChannelKey`	string | null	Channel to resume
`favorites`	string[]	Channel keys
`filter`	`{search, favOnly, country, category}`	Last filter state
`seenWelcome`	boolean	Welcome card already shown
`recentlyWatched`	`{key, url, name, logo, watchedAt}[]`	Newest first, at most 15

6.7 Constants

Name	Value	Where	Purpose
`DEFAULT_URL`	`https://iptv-org.github.io/iptv/index.m3u`	page	Built-in playlist
`REFRESH_MS`	6 h	page	Background refresh age
`BUFFER`	6 rows	page	Extra rows rendered above and below the viewport
`LONG_PRESS`	550 ms	page	Hold time for options
`STRIP_HIDE`	8 s	page	Channel strip auto-hide
`CHROME_HIDE`	5 s	page	Clock and hint auto-hide
`RECENT_MAX`	15	page	History length
`TEXT_LIMIT`	1,500,000 chars	page	Largest local playlist stored in `localStorage`
`MIN_WEBVIEW`	62	Kotlin	Oldest supported WebView major version
`USER_AGENT`	`MZIPTV/0.1 (Android TV)`	Kotlin	Default request User-Agent
Connect / read timeout	10 s / 15 s (streams), 15 s / 30 s (playlists)	Kotlin	Network limits

6.8 Glossary

Term	Meaning
ExoPlayer / Media3	Google's Android media player library
DASH	Dynamic Adaptive Streaming over HTTP: like HLS, with an `.mpd` manifest; plays in the app only
HLS	HTTP Live Streaming: video split into short segments listed in an `.m3u8` playlist

Term	Meaning
hls.js	JavaScript library that plays HLS in browsers without native support
M3U / M3U8	Plain-text playlist format; M3U8 is the UTF-8 variant
`tvg-id`	Channel identifier in IPTV playlists, e.g. `CNN.us`
WebView	Android component that renders web pages inside an app
Bridge	The pair of mechanisms that let page and Kotlin call each other
CORS	Browser rule that blocks reading responses from another origin unless the server allows it
Live edge	The newest moment of a live stream
Overscan	TVs cropping the edges of the picture
Virtualization	Rendering only the visible part of a long list
Strategy pattern	Interchangeable objects behind one interface, chosen at runtime
dp	Android density-independent pixel; equals one CSS px in a full-window WebView

Appendix

A. Third-party components

Component	Version	Licence	Where
hls.js	1.5.13	Apache 2.0	`web/vendor/hls.min.js`
Inter variable font	5.1.0	SIL Open Font License 1.1	`web/vendor/inter-latin-wght-normal.woff2`
AndroidX Media3 (ExoPlayer, HLS, DASH, UI)	1.5.1	Apache 2.0	Gradle dependency
AndroidX WebKit	1.12.1	Apache 2.0	Gradle dependency

Component	Version	Licence	Where
AndroidX Activity KTX	1.9.3	Apache 2.0	Gradle dependency
iptv-org playlist	Rebuilt daily	Public channel links collected by iptv-org	Downloaded at run time
Playwright	1.64.1	Apache 2.0	Dev dependency for the tests only (not shipped)

The app itself is MIT-licensed, © 2026 Maze Media.

B. Revision history

Date	Doc version	App version	Change
2026-10-26	1.0	0.1.0	First edition: manual, architecture, fundamentals, advanced techniques, developer guide, reference
2026-10-26	1.1	0.1.0	User Manual expanded to 15 sections with 14 screenshots